
MA 578: BAYESIAN STATISTICS

Chapter 7: Markov Chain Monte Carlo

AmirEmad Ghassami

Department of Mathematics and Statistics

Boston University

Fall 2025

1 Introduction

Thus far we have studied basic Monte Carlo schemes—e.g., rejection sampling and importance sampling—that produce *independent* draws from a target distribution. While conceptually simple, these methods can be fragile or inefficient in high dimensions (e.g., low acceptance in rejection sampling; high-variance weights in importance sampling).

Markov chain Monte Carlo (MCMC) methods address this by generating a *dependent* sequence of samples via a Markov chain on a state space $\mathcal{Z}$ whose *stationary* (invariant) distribution is the target $\pi(z)$. The goal is to design the chain so that the time averages converge to expectations under π :

$$\frac{1}{S} \sum_{s=1}^S f(Z^{(s)}) \xrightarrow[S \rightarrow \infty]{\text{a.s.}} \mathbb{E}_{\pi}[f(Z)].$$

In practice, MCMC delivers accurate approximations even in large dimensions where i.i.d. sampling is infeasible, at the cost of serial dependence—hence the need to reason about burn-in, mixing, and effective sample size. MCMC is a general and widely used framework (the Metropolis algorithm was named among SIAM’s “Top 10 Algorithms of the 20th Century”).

We begin this chapter with a brief review of the main concepts in Markov chains.

2 Markov Chains

Definition 2.1 (First-order Markov chain). A sequence of random variables $\{Z^{(s)}\}_{s \geq 1}$ on a state space $\mathcal{Z}$ is a first-order Markov chain if for every $s \geq 1$ and all values with $p(Z^{(1)} = z^{(1)}, \dots, Z^{(s)} = z^{(s)}) > 0$,

$$p(Z^{(s+1)} = z^{(s+1)} \mid Z^{(1)} = z^{(1)}, \dots, Z^{(s)} = z^{(s)}) = p(Z^{(s+1)} = z^{(s+1)} \mid Z^{(s)} = z^{(s)}).$$

It is called *homogeneous* if the one-step law does not depend on s , i.e., there exists a transition function $T(z, z')$ such that

$$p(Z^{(s+1)} = z' \mid Z^{(s)} = z) = T(z, z') \quad \text{for all } s.$$

A homogeneous first-order Markov chain can be specified with the distribution of the initial state $p_1(z)$, together with the transition function T :

$$p(Z^{(1)} = z^{(1)}, \dots, Z^{(S)} = z^{(S)}) = p_1(z^{(1)}) \prod_{s=2}^S T(z^{(s-1)}, z^{(s)}).$$

Finite-state Markov chain. If $\mathcal{Z} = \{1, \dots, K\}$, the transition function is a *stochastic matrix* $A \in [0, 1]^{K \times K}$ with row sums one:

$$A_{ij} = T(i, j) = \Pr(Z^{(s)} = j \mid Z^{(s-1)} = i), \quad \sum_{j=1}^K A_{ij} = 1.$$

In this case, the Markov chain can be represented visually by the *state transition diagram*.

A two-state example with parameters $\alpha, \beta \in [0, 1]$:

$$A = \begin{bmatrix} 1 - \alpha & \alpha \\ \beta & 1 - \beta \end{bmatrix}. \quad \begin{array}{c} \alpha \\ \text{1} \quad \text{2} \\ \beta \end{array}$$

Stationary (invariant) distribution. A distribution π on $\mathcal{Z}$ is *stationary* for the chain if

$$\pi(z') = \int_{\mathcal{Z}} \pi(z) T(z, z') dz.$$

Integral becomes a sum on discrete $\mathcal{Z}$:

$$\pi(z') = \sum_z \pi(z) T(z, z').$$

π is called stationary because it has the following property: if $Z^{(s)} \sim \pi$ and $Z^{(s+1)} \mid Z^{(s)} \sim T(Z^{(s)}, \cdot)$, then $Z^{(s+1)} \sim \pi$. In words, if $Z^{(s)} \sim \pi$ and $Z^{(s+1)}$ is generated from the Markov chain, then $Z^{(s+1)} \sim \pi$. That is, once we are sampling from π , we are always sampling from π .

Remark. A chain may have multiple stationary distributions. For example, if T is the identity transformation, then any distribution will be a stationary distribution.

Definition 2.2 (Detailed balance). A distribution p satisfies *detailed balance* (DB) with respect to T if

$$p(z) T(z, z') = p(z') T(z', z) \quad \text{for all } z, z' \in \mathcal{Z}.$$

Intuition. For each pair (z, z') , the probability flux from z to z' equals the flux back from z' to z .

Proposition 2.1. p satisfying the detailed balance property with respect to T is a sufficient (but not necessary) condition for ensuring that p is a stationary distribution for the chain.

Proof. Using DB,

$$\sum_{z'} p(z') T(z', z) = \sum_{z'} p(z) T(z, z') = p(z) \sum_{z'} T(z, z') = p(z),$$

and the analogous integral argument applies on general spaces. $\square$

Uniqueness and convergence: irreducibility, aperiodicity, recurrence. We recall three structural properties:

- *Irreducible*: every state can reach every other with positive probability in some number of steps.
Counterexample: Consider a Markov chain on integers with a transition function that only allows it to go to $z^{(s)} \pm 2$ from $z^{(s)}$. This Markov chain can only generate even numbers if $z^{(1)}$ is even and only odd numbers if $z^{(1)}$ is odd.
- *Aperiodic*: a state z has period $d \geq 1$ if all return times to z are multiples of d ; a chain is aperiodic if every state has period 1 (no deterministic cycling).
- *Recurrent*: a state z is *recurrent* if when we continue running the Markov chain from z , we are guaranteed to eventually return to z . A Markov chain with all states recurrent is called recurrent.
For finite state spaces, irreducible $\Rightarrow$ all states are recurrent.

Theorem 2.2 (Ergodic theorem for Markov chains). If a homogeneous Markov chain is irreducible, aperiodic, and recurrent, then:

1. There exists a unique stationary distribution π .
2. For any initial distribution and any measurable set A ,

$$\Pr(Z^{(s)} \in A) \rightarrow \pi(A) \quad \text{as } s \rightarrow \infty.$$

3. For any π -integrable f ,

$$\frac{1}{S} \sum_{s=1}^S f(Z^{(s)}) \xrightarrow[S \rightarrow \infty]{\text{a.s.}} \int f(z) \pi(z) dz = \mathbb{E}_\pi[f(Z)].$$

3 Markov Chain Monte Carlo (MCMC)

Basic idea. Construct a Markov chain on the state space $\mathcal{Z}$ with transition function $T(z, z')$ whose *stationary* (invariant) distribution is the target $\pi(z)$. In Bayesian inference, the target is often a posterior $p(\theta \mid y)$.

Independent MC vs. MCMC. In order to approximate integrals, we need the empirical distribution of the samples to approach the target distribution. In basic Monte Carlo, we draw $Z^{(s)} \stackrel{\text{i.i.d.}}{\sim} \pi$ so that for all s and all measurable A ,

$$\Pr(Z^{(s)} \in A) = \pi(A).$$

In MCMC, our only guarantee is asymptotic: the marginal law of $Z^{(s)}$ typically depends on s and only converges to π as $s \rightarrow \infty$. Specifically, as we saw in Theorem 2.2, under certain conditions (irreducible, aperiodic, recurrent), two consequences hold:

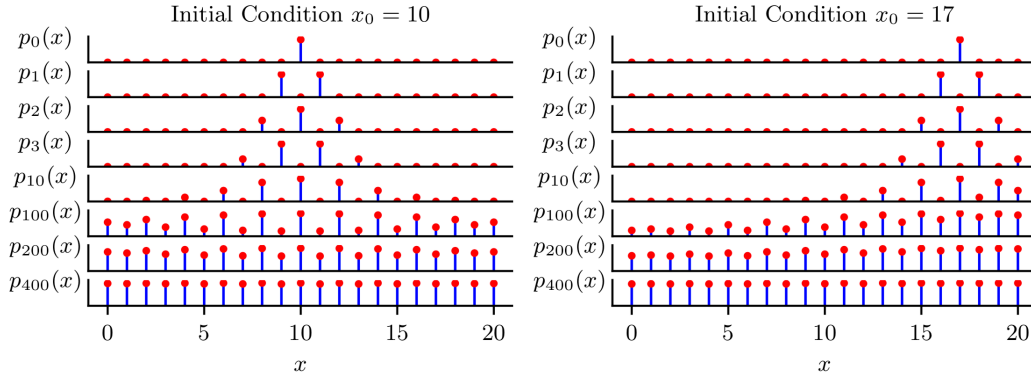
$$\Pr(Z^{(s)} \in A) \xrightarrow{s \rightarrow \infty} \pi(A) \quad \text{and} \quad \frac{1}{S} \sum_{s=1}^S f(Z^{(s)}) \xrightarrow[S \rightarrow \infty]{\text{a.s.}} \mathbb{E}_\pi[f(Z)].$$

In other words, the samples will be coming from the chain’s stationary distribution only when the chain has “forgotten” where it started from.

Thus MCMC trades independence for dependence but attains consistent Monte Carlo integration when i.i.d. sampling is infeasible.

Example. Let the target distribution be $\pi = \text{Unif}\{1, 2, \dots, 20\}$. Suppose we sample using symmetric random walk that proposes ± 1 (with reflection at the boundaries). Different initializations can require hundreds of steps before the marginal law is close to π ; the retained post–burn-in draws then approximate uniformity.

Two runs of the algorithm:



It can be seen that even in this small problem it takes over 200 steps until the chain has “forgotten” where it started from.

Mixing and burn-in. Mixing time is the amount of time it takes for a Markov chain to enter the stationary distribution. Mixing time depends on the initial value and on how the Markov chain is designed (e.g., geometry-aware updates). Reducing the mixing time is one of the most important factors in making the algorithm fast.

In practice, we run the Markov chain as long as feasible and discard the initial part of the chain; the part that *has a trend*. This initial part of the Markov chain is called the *burn-in period*. Formally, we discard an initial *burn-in* segment $\{Z^{(1)}, \dots, Z^{(s_0)}\}$ to remove transient behavior, and retain $\{Z^{(s_0+1)}, \dots, Z^{(S)}\}$ for inference:

$$\widehat{\mathbb{E}_\pi[f]} = \frac{1}{S - s_0} \sum_{s=s_0+1}^S f(Z^{(s)}).$$

Therefore, the procedure is as follows:

- Suppose our target distribution is $p(z)$
- We design a Markov chain such that its stationary distribution is $\pi = p$
- We run the Markov chain for a while. The values it will produce after the burn-in period approximately have distribution p

We will study three algorithms for designing the Markov chain:

- **Metropolis–Hastings (MH):** accept/reject correction of a proposal kernel to enforce invariance (often via detailed balance).
- **Gibbs sampling:** coordinate-wise updates from full conditionals (special case of MH with acceptance probability 1).
- **Hamiltonian Monte Carlo (HMC):** geometry-informed proposals using Hamiltonian dynamics for high-dimensional targets.

There are two key questions when using MCMC:

- How large should the burn-in period s_0 be?
- How large S should be?

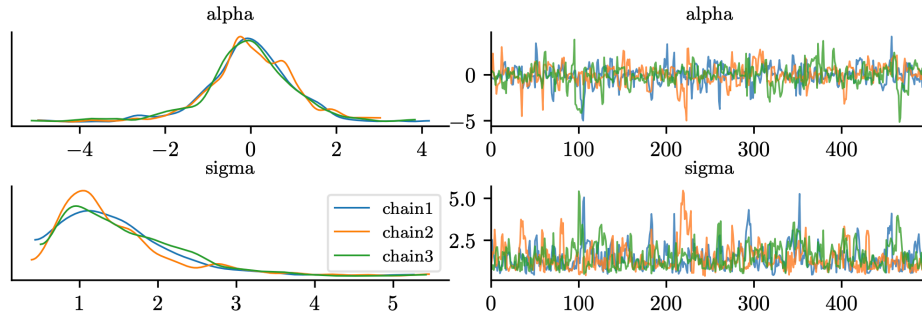
The first question is very hard. Because in practice, it can be difficult to compute the mixing time theoretically. There is no universal theory to compute the mixing time for a given problem (a practical limitation of MCMC). Nevertheless, there exist heuristics to check for mixing. These are referred to as *MCMC convergence diagnostic methods*. In general, we cannot know for sure if your chain has converged. But sometimes you can know if your chain has not converged.

The main heuristic is based on the *trace plots* and the *autocorrelation function plot*.

3.1 Trace plots

A trace plot displays sampled values versus iteration index for one or more chains. Trace plot can also be a plot of the samples of some quantity of interest, such as the value of a certain component of the state vector, or any scalar function of the state, or some event such as the value taking on an extreme value.

Setup. Run multiple chains (typically 3–4) from deliberately overdispersed initial values to probe different regions of the target.

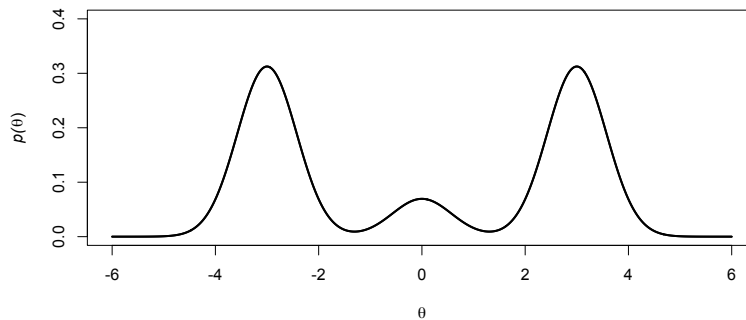


What to look for.

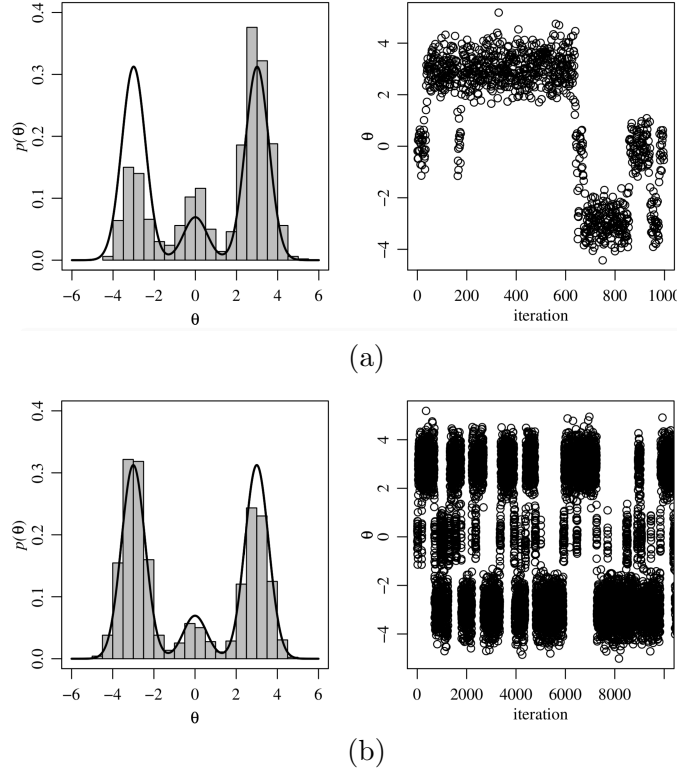
- **Stationarity (no visible trend).** Within each chain, the series should fluctuate around a stable level without long drifts. Across chains, the marginal behavior over time should look similar.
- **Between-chain mixing (overlap).** After burn-in, traces from different chains should overlap and explore the same regions; lack of overlap signals dependence on initialization.
- **Autocorrelation / stickiness.** Highly persistent “flat” stretches or slow movement indicate strong serial correlation and poor mixing. Rapid back-and-forth exploration suggests lower autocorrelation.
- **Mode exploration.** For multimodal targets, check that chains visit all relevant modes with reasonable frequency; trace plots reveal mode-sticking.

If the chain has mixed, post-burn-in segments of different chains appear exchangeable and overlapping. In contrast, traces that get “stuck” in regions (e.g., at distinct modes) reveal high autocorrelation (correlation between consecutive values of the chain) and inadequate movement between regions, leading to poor finite-sample approximations even when the stationary distribution is correct asymptotically.

Example. Consider a sampler targeting the three-modal density in the figure below:



With 1,000 MCMC draws, the histogram in panel (a) below poorly matches the target distribution because the chain spends long stretches in a single mode; the trace plot shows the θ values get “stuck” in certain regions and we have infrequent jumps between modes (high autocorrelation). Nevertheless, the chain eventually provide a good approximation to the target distribution as seen in Part (b) of the figure.



Remark. Trace plots are heuristic: they can reveal non-convergence, but an apparently stationary trace is not a proof of convergence. Use them alongside quantitative diagnostics (e.g., autocorrelation and effective sample size) discussed next.

Example: Auto-Regressive Process. Auto-regressive process, $AR(1)$, is defined as

$$Z^{(s)} = \alpha Z^{(s-1)} + \epsilon^{(s)},$$

for iid, mean-zero variables $\{\epsilon^{(s)}\}$, and for some $Z^{(1)} \in \mathbb{R}$. We assume $|\alpha| < 1$, $\epsilon^{(s)} \sim \mathcal{N}(0, \sigma_0^2)$, with σ_0^2 known.

Suppose $z^{(s)} \sim \mathcal{N}(0, \nu^2)$. Then

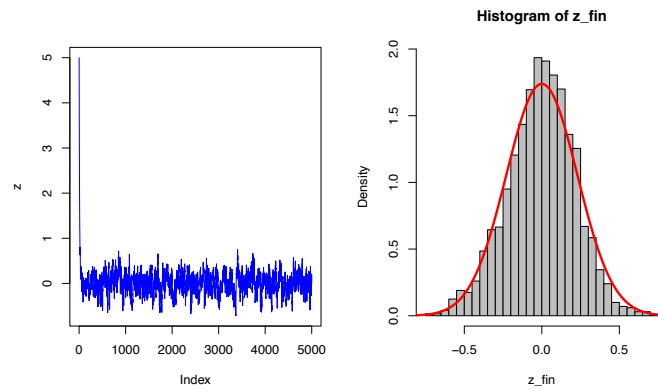
$$z^{(s+1)} = \alpha z^{(s)} + \sigma_0 N \sim \mathcal{N}(0, \alpha^2 \nu^2 + \sigma_0^2).$$

Solving $\alpha^2 \nu^2 + \sigma_0^2 = \nu^2$, we obtain $\nu^2 = \frac{\sigma_0^2}{1 - \alpha^2}$. Therefore, the stationary distribution for this $AR(1)$ Markov chain is

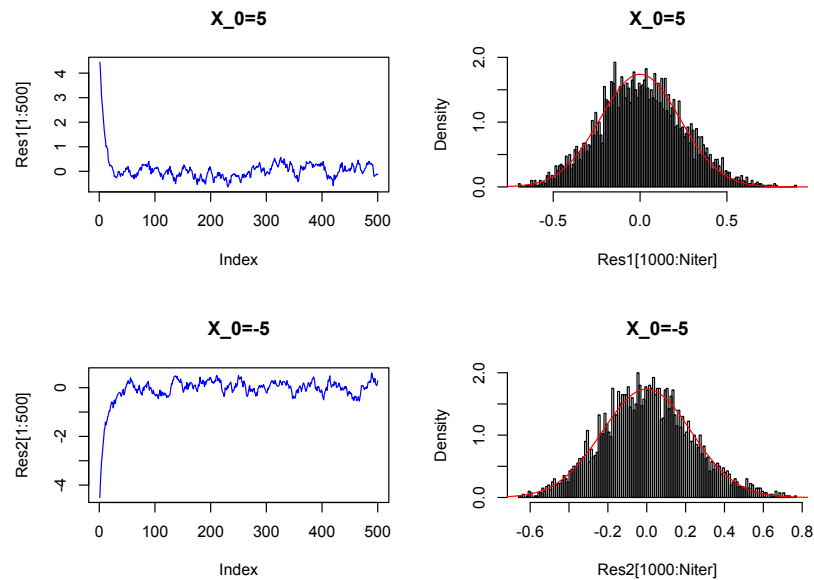
$$\pi = \mathcal{N}\left(0, \frac{\sigma_0^2}{1 - \alpha^2}\right).$$

Code for the $AR(1)$ Markov chain:

```
alpha <- 0.9
sigma_0 <- 0.1
Niter <- 1000
Res <- numeric(Niter)
z <- 5
for (j in 1:Niter){
  z <- alpha*z + sigma_0*rnorm(1)
  Res[j] <- z
}
```



No matter how we choose $z^{(1)}$, after running the Markov chain for a while, the values $z^{(s)}, z^{(s+1)}, \dots$ will all have a distribution that is approximately π . Therefore, for large S , we get samples from π .



3.2 Effective Sample Size

Suppose we are interested in approximating the parameter $\psi = \mathbb{E}_\pi[f(Z)]$ via finite sample

$$\hat{\psi} = \frac{1}{S} \sum_{s=1}^S f(Z^{(s)}),$$

where $\{Z^{(s)}\}$ are post-burn-in draws from a Markov chain with stationary distribution π . That is, samples are from the target distribution, but they are not independent. The challenge is that the information in S MCMC samples is not the same as the information in S independent samples. Therefore, the natural question is: How should we compute the *effective sample size*?

To make the notation simpler, let $Y^{(s)} = f(Z^{(s)})$ and write $\sigma_f^2 = \text{var}_\pi(Y^{(s)})$. If the $Y^{(s)}$ were i.i.d., then

$$\text{var}(\hat{\psi}) = \frac{\sigma_f^2}{S}.$$

Claim. Let $\{Y^{(s)}\}_{s=1}^S$ have finite second moments and be second-order stationary: $\mathbb{E}(Y^{(s)}) = \mu$ and $\text{cov}(Y^{(i)}, Y^{(j)}) = \gamma_{|i-j|}$ for all i, j with $\gamma_0 = \text{var}(Y^{(s)}) < \infty$. Then the finite-sample variance is

$$\text{var}(\hat{\psi}) = \text{var}\left(\frac{1}{S} \sum_{s=1}^S Y^{(s)}\right) = \frac{1}{S} \left[\gamma_0 + 2 \sum_{\ell=1}^{S-1} \left(1 - \frac{\ell}{S}\right) \gamma_\ell \right], \quad \gamma_\ell = \text{cov}(Y^{(s)}, Y^{(s+\ell)}), \quad (1)$$

which reduces to the i.i.d. case when $\gamma_\ell = 0$ for $\ell \geq 1$.

Proof.

Step 1: Expand var of the average.

$$\text{var}(\hat{\psi}) = \text{var}\left(\frac{1}{S} \sum_{s=1}^S Y^{(s)}\right) = \frac{1}{S^2} \text{var}\left(\sum_{s=1}^S Y^{(s)}\right) = \frac{1}{S^2} \sum_{i=1}^S \sum_{j=1}^S \text{cov}(Y^{(i)}, Y^{(j)}).$$

Step 2: Use stationarity and count pairs by lag. By stationarity, $\text{Cov}(Y^{(i)}, Y^{(j)}) = \gamma_{|i-j|}$. Split the double sum into the diagonal ($i = j$) and off-diagonals ($i \neq j$):

$$\sum_{i=1}^S \sum_{j=1}^S \text{cov}(Y^{(i)}, Y^{(j)}) = \underbrace{\sum_{i=1}^S \gamma_0}_{\text{diagonal}} + \underbrace{\sum_{i \neq j} \gamma_{|i-j|}}_{\text{off-diagonal}}.$$

For each fixed $\ell \in \{1, \dots, S-1\}$, the pairs with $j - i = \ell$ are

$$(i, j) = (1, 1 + \ell), (2, 2 + \ell), \dots, (S - \ell, S),$$

which gives exactly $S - \ell$ pairs. By symmetry there are also $S - \ell$ pairs with $i - j = \ell$. Hence, for each $\ell \geq 1$, the off-diagonal term contributes $2(S - \ell)\gamma_\ell$.

Step 3: Combine and simplify.

$$\sum_{i=1}^S \sum_{j=1}^S \text{cov}(Y^{(i)}, Y^{(j)}) = S\gamma_0 + 2 \sum_{\ell=1}^{S-1} (S - \ell)\gamma_\ell,$$

and therefore

$$\text{var}(\hat{\psi}) = \frac{1}{S^2} \left[S\gamma_0 + 2 \sum_{\ell=1}^{S-1} (S - \ell)\gamma_\ell \right] = \frac{1}{S} \left[\gamma_0 + 2 \sum_{\ell=1}^{S-1} \left(1 - \frac{\ell}{S}\right) \gamma_\ell \right],$$

which is exactly (1). The factor $(1 - \ell/S)$ is the normalized count of index pairs at lag ℓ . $\square$

Define the lag- ℓ autocorrelation

$$\rho_\ell = \text{corr}(Y^{(s)}, Y^{(s+\ell)}) = \frac{\text{cov}(Y^{(s)}, Y^{(s+\ell)})}{\sqrt{\text{var}(Y^{(s)})\text{var}(Y^{(s+\ell)})}} = \frac{\gamma_\ell}{\gamma_0},$$

where $\sigma_f^2 = \text{var}_\pi(Y^{(s)}) = \gamma_0$, and we used the fact that we are drawing samples from the target distribution, and hence, for all s , we have $\sigma_f^2 = \text{var}_\pi(Y^{(s)})$.

Starting from (1), write $\gamma_\ell = \sigma_f^2 \rho_\ell$, we have

$$\text{var}(\hat{\psi}) = \frac{\sigma_f^2}{S} \left[1 + 2 \sum_{\ell=1}^{S-1} \left(1 - \frac{\ell}{S}\right) \rho_\ell \right]. \quad (2)$$

Proposition 3.1. Assume the post-burn-in sequence $\{Y^{(s)}\}$ is second-order stationary and $\sum_{\ell=1}^{\infty} |\rho_\ell| < \infty$. Then

$$S \text{var}(\hat{\psi}) \longrightarrow \sigma_f^2 \tau_f, \quad \text{equivalently} \quad \text{var}(\hat{\psi}) = \frac{\sigma_f^2}{S} \tau_f + o\left(\frac{1}{S}\right), \quad (3)$$

where $\tau_f = 1 + 2 \sum_{\ell=1}^{\infty} \rho_\ell$ is the integrated autocorrelation time (IACT).

As a sanity check, with i.i.d. draws, $\rho_\ell = 0$ for $\ell \geq 1 \Rightarrow \tau_f = 1$ and $\text{var}(\hat{\psi}) = \sigma_f^2/S$.

Proof. Starting from (2),

$$\text{var}(\hat{\psi}) = \frac{\sigma_f^2}{S} \left[1 + 2 \sum_{\ell=1}^{S-1} \rho_\ell \right] - \frac{2\sigma_f^2}{S^2} \sum_{\ell=1}^{S-1} \ell \rho_\ell.$$

Step 1 (truncate to the infinite sum). Since $\sum_{\ell \geq 1} |\rho_\ell| < \infty$, $R_S := \sum_{\ell=S}^{\infty} \rho_\ell \rightarrow 0$, and thus

$$\frac{\sigma_f^2}{S} \left[1 + 2 \sum_{\ell=1}^{S-1} \rho_\ell \right] = \frac{\sigma_f^2}{S} \tau_f - \frac{2\sigma_f^2}{S} R_S = \frac{\sigma_f^2}{S} \tau_f + o\left(\frac{1}{S}\right).$$

Step 2 (edge-weight term is $o(1/S)$). Write

$$\frac{1}{S^2} \sum_{\ell=1}^{S-1} \ell \rho_\ell = \frac{1}{S} \left[\frac{1}{S} \sum_{\ell=1}^{S-1} \ell \rho_\ell \right].$$

Because $\sum_{\ell \geq 1} |\rho_\ell| < \infty$, the tail sums $a_m := \sum_{\ell=m}^{\infty} |\rho_\ell|$ satisfy $a_m \downarrow 0$. Then

$$\frac{1}{S} \sum_{\ell=1}^{S-1} \ell |\rho_\ell| = \frac{1}{S} \sum_{m=1}^{S-1} \sum_{\ell=m}^{S-1} |\rho_\ell| \leq \frac{1}{S} \sum_{m=1}^{S-1} a_m \xrightarrow{S \rightarrow \infty} 0$$

(by Cesàro's lemma). Hence $\frac{1}{S^2} \sum_{\ell=1}^{S-1} \ell \rho_\ell = o(1/S)$, and the edge term $\frac{2\sigma_f^2}{S^2} \sum_{\ell=1}^{S-1} \ell \rho_\ell = o(1/S)$.

Combining the two steps yields $\text{var}(\hat{\psi}) = \frac{\sigma_f^2}{S} \tau_f + o(1/S)$, which is equivalent to $S \text{var}(\hat{\psi}) \rightarrow \sigma_f^2 \tau_f$.

□

What the weights $(1 - \ell/S)$ are doing. In the finite- S formula, $(1 - \ell/S)$ is the fraction of index pairs at lag ℓ among the S^2 total pairs. Proposition 3.1 shows that, under short-range dependence, these edge corrections vanish at rate $1/S$, leaving the *limit* governed by the infinite-lag sum of correlations.

Remark (Long-range dependence). If $\sum_{\ell \geq 1} |\rho_\ell| = \infty$ (e.g., $\rho_\ell \asymp c \ell^{-\beta}$ with $\beta \leq 1$), then τ_f diverges and $\text{var}(\hat{\psi})$ can decay slower than $1/S$; the standard Markov chain CLT may fail.

Effective sample size (ESS). Equation (3) motivates the definition

$$S_{\text{eff}}(f) = \frac{S}{\tau_f}, \quad \text{so that} \quad \text{var}(\hat{\psi}) = \frac{\sigma_f^2}{S_{\text{eff}}(f)}.$$

Thus S_{eff} is the number of i.i.d. draws from π that would deliver the same precision for estimating ψ . Some software caps $S_{\text{eff}} \leq S$ for reporting, although mathematically $S_{\text{eff}} > S$ is possible when $\tau_f < 1$.

By the Markov chain CLT: $\sqrt{S}(\hat{\psi} - \psi) \Rightarrow \mathcal{N}(0, \sigma_f^2 \tau_f)$ under standard mixing conditions.

We see that

$$\text{var}_{\text{MCMC}}(\hat{\psi}) = \text{var}_{\text{indep}}(\hat{\psi}) \cdot \tau_f,$$

$$\Rightarrow \frac{1}{\tau_f} = \frac{\text{var}_{\text{indep}}(\hat{\psi})}{\text{var}_{\text{MCMC}}(\hat{\psi})}$$

Intuitively, $1/\tau_f$ is the number of independent samples necessary to give the same precision as the MCMC samples. If τ_f is very large, then the Markov chain moves around the state space very slowly. The higher the IACT, the more MCMC samples we need to attain a given level of precision for our approximation

Estimating IACT and the Monte Carlo standard error (MCSE). The auto-correlation function ρ_ℓ can be estimated from data. Let $\bar{Y} = \frac{1}{S} \sum_{s=1}^S Y^{(s)}$. The sample auto-correlation function is

$$\hat{\rho}_\ell = \frac{\frac{1}{S-\ell} \sum_{s=1}^{S-\ell} (Y^{(s)} - \bar{Y})(Y^{(s+\ell)} - \bar{Y})}{\frac{1}{S} \sum_{s=1}^S (Y^{(s)} - \bar{Y})^2}.$$

Moreover, we can estimate the variance with $\hat{\sigma}_f^2 = \frac{1}{S-1} \sum_{s=1}^S (Y^{(s)} - \bar{Y})^2$, and estimate the variance of the sampling distribution as

$$\widehat{\text{var}}(\hat{\psi}) = \frac{\hat{\tau}_f \hat{\sigma}_f^2}{S}.$$

This leads to estimated MC standard error

$$\widehat{\text{MCSE}}(\hat{\psi}) = \sqrt{\widehat{\text{var}}(\hat{\psi})} = \sqrt{\frac{\hat{\sigma}_f^2}{S}} \hat{\tau}_f,$$

which yields approximate $(1 - \alpha)$ confidence intervals,

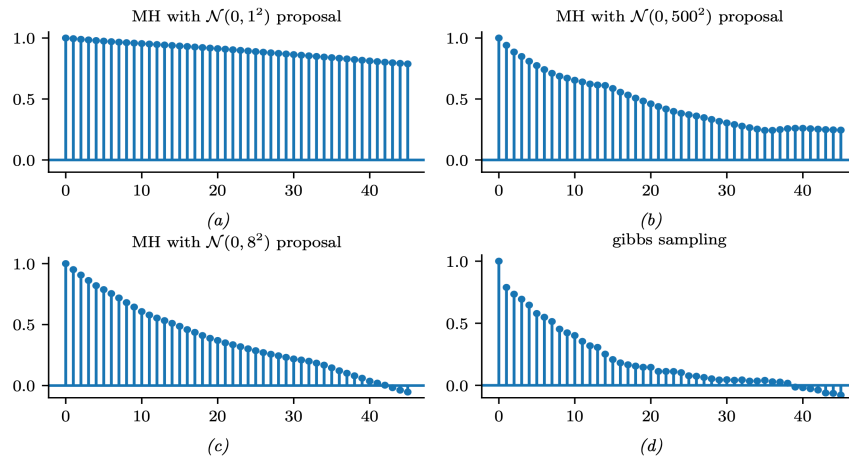
$$\hat{\psi} \pm z_{1-\alpha/2} \widehat{\text{MCSE}}(\hat{\psi}).$$

Example: AR(1) time series. For the stationary Gaussian AR(1) process $Y^{(s)} = \alpha Y^{(s-1)} + \epsilon^{(s)}$ with $|\alpha| < 1$ and $\epsilon^{(s)} \sim \mathcal{N}(0, \sigma_\epsilon^2)$, the lag- ℓ autocorrelation is $\rho_\ell = \alpha^\ell$, hence

$$\tau_f = 1 + 2 \sum_{\ell=1}^{\infty} \alpha^\ell = \frac{1+\alpha}{1-\alpha}, \quad S_{\text{eff}} = S \frac{1-\alpha}{1+\alpha}.$$

Positive persistence ($\alpha > 0$) shrinks S_{eff} , whereas negative persistence ($\alpha < 0$) can increase it.

Diagnose via ACF plots. Use the ACF of $Y^{(s)} = f(Z^{(s)})$ to diagnose *mixing and Monte Carlo efficiency*: slow decay of $\hat{\rho}_\ell$ implies large $\hat{\tau}_f$, hence small $S_{\text{eff}} = S/\hat{\tau}_f$ and a larger MCSE; fast decay indicates better mixing and larger S_{eff} . (ACF is a heuristic—combine with trace plots and between-chain checks.) See figure below.



Practical notes.

- *Thinning* means retaining only every k -th MCMC draw and discarding the others:

$$\tilde{Y}^{(j)} = Y^{(1+(j-1)k)}, \quad j = 1, \dots, \left\lfloor \frac{S}{k} \right\rfloor,$$

for some integer $k \geq 2$. Practitioners sometimes thin to “reduce autocorrelation” in the saved draws or to save disk space.

Thinning rarely helps ESS. Thinning reduces storage but usually *decreases* S_{eff} for a fixed compute budget. Prefer to keep all post-burn-in draws and summarize uncertainty via ESS/MCSE. If draws are highly autocorrelated, improve the *sampler* (e.g., reparameterize, tune step sizes) to reduce τ rather than discarding samples.

- **Function-specific ESS.** All formulas apply with $Y^{(s)} = f(Z^{(s)})$. Report S_{eff} for key scalar summaries (means, variances, regression coefficients, log-likelihood, etc.).
- **Software.** Sample ACFs are implemented in common libraries (e.g., `acf()` in R).

4 Metropolis–Hastings Algorithm

In this section we introduce Metropolis–Hastings (MH), a fundamental way to *construct* a Markov chain whose stationary distribution is a given target distribution $p(z) = \tilde{p}(z)/C$, where C is an unknown normalizing constant.

Basic procedure.

- At each step, from the current state z , draw a *proposal* $z' \sim q(z' | z)$, where q is called the *proposal distribution*.
- Decide to *accept* or *reject* z' using an *acceptance probability*.

- If accepted, move to z' ; if rejected, stay at z (i.e., the next state repeats the current value).

The choice of the acceptance probability is what makes p the stationary distribution.

4.1 Metropolis Algorithm (symmetric proposal)

We start with the Metropolis algorithm in which the proposal is required to be *symmetric*, that is, $q(z' | z) = q(z | z')$. Then the Metropolis acceptance probability is

$$A(z' | z) = \min \left\{ 1, \frac{\tilde{p}(z')}{\tilde{p}(z)} \right\}.$$

Note that only the *unnormalized* target $\tilde{p}$ is required; the normalizing constant cancels.

Algorithm 1 Metropolis Algorithm

```

Initialize:  $z^{(0)}$ 
for  $s = 0, 1, \dots$  do
   $z \leftarrow z^{(s)}$ 
  Propose  $z' \sim q(z' | z)$ 
  Compute  $A = \min \left\{ 1, \frac{\tilde{p}(z')}{\tilde{p}(z)} \right\}$ 
  Draw  $U \sim \text{Unif}(0, 1)$ 
  if  $U \leq A$  then
     $z^{(s+1)} \leftarrow z'$  ▷ accept
  else
     $z^{(s+1)} \leftarrow z$  ▷ reject
  end if
end for

```

If z' has higher target density than z , the move is always accepted; this leads to “uphill” moves. If it is lower, the move can still be accepted with probability proportional to the density ratio. *Unlike rejection sampling*, a rejected proposal is not discarded: we record the current state again, so every iteration yields a draw; moreover, the acceptance rule sometimes admits “downhill” moves to lower-density states, and together these features ensure the chain has the correct stationary distribution (i.e., the right long-run frequencies under p).

Intuition. (Formal justification via detailed balance appears later.) Suppose we are at state z (i.e., z is already in the set of our samples).

- If z' has higher probability, we should include it because we have already included something with lower probability (i.e., z).

- If z' has lower probability, we would like the relative frequency of z' to z be equal to $A = \frac{p(z')}{p(z)}$ which is the ratio of these two states in our target distribution. Therefore, we accept z' with probability A .

Remarks.

- The samples $z^{(1)}, z^{(2)}, \dots$ are *dependent*; successive states are typically highly auto-correlated.
- In Bayesian inference,

$$\frac{p(z')}{p(z)} \rightarrow \frac{p(\theta' | y)}{p(\theta | y)} = \frac{p(y | \theta') p(\theta')}{p(y | \theta) p(\theta)},$$

so sampling model and prior enter multiplicatively.

- Numerical stability: compute on the log scale,

$$\log A = \min\{0, [\log \tilde{p}(z') - \log \tilde{p}(z)]\}.$$

4.1.1 Gaussian Random-Walk Metropolis Algorithm

Let the target be $p(z) \propto \tilde{p}(z)$ on $\mathbb{R}$. In Gaussian random walk Metropolis algorithm, the proposal distribution is

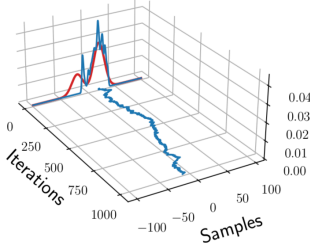
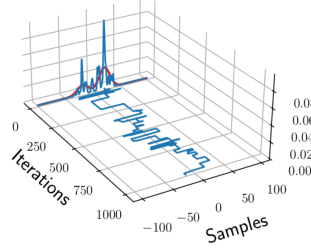
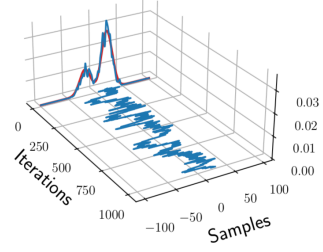
$$q(z' | z) = \mathcal{N}(z' | z, \sigma^2), \quad \text{i.e., } z' = z + \sigma N, \quad N \sim \mathcal{N}(0, 1).$$

We choose a scale $\sigma > 0$ and an initial value $z^{(0)}$, then run the Metropolis algorithm with the Gaussian proposal above. At each step, the algorithm proposes a new state $z + \sigma N$, where $N \sim \mathcal{N}(0, 1)$. If the density is higher at $z + \sigma N$, then we move to that location ($z^{(s+1)} = z + \sigma N$). Otherwise, we may still move to $z + \sigma N$, with a probability proportional to the ratio $p(z + \sigma N)/p(z)$.

Tuning the scale σ is crucial:

- If σ is too small, steps are tiny: very high acceptance but poor exploration (the chain lingers within a narrow region and mixes slowly).
- If σ is too large, proposals often land in low-density regions: very low acceptance, creating long “sticky” stretches of repeated states.
- A well-tuned σ balances move size and acceptance to explore effectively. (Rules of thumb: in $d = 1$ aim for acceptance ≈ 0.4 – 0.5 ; in higher d , around ≈ 0.2 – 0.3 is often reasonable.)

The figure below illustrates sampling from a mixture of two Gaussians with different choices of σ . Too small a variance traps the chain in one mode; too large yields many rejections; a moderate choice crosses modes and reproduces the mixture weights.

MH with $\mathcal{N}(0, 1^2)$ proposalMH with $\mathcal{N}(0, 500^2)$ proposalMH with $\mathcal{N}(0, 8^2)$ proposal

Remark 1. The extension to $\mathbb{R}^d$ replaces the proposal by $q(z' | z) = \mathcal{N}(z' | z, \Sigma)$, commonly with $\Sigma = s^2 I_d$ or a preconditioned covariance $\Sigma = s^2 \widehat{\text{Cov}}$ to better match the target's geometry.

Interactive demos: <http://chi-feng.github.io/mcmc-demo/app.html>.

Remark 2 (log-scale computation). It is numerically stable to evaluate ratios as

$$\exp\left([\log \tilde{p}(z') - \log \tilde{p}(z)]\right).$$

Example 1. Normal model with known variance.

- Prior: $\theta | \sigma^2 \sim \mathcal{N}(\mu, \tau^2)$
- Sampling model: $Y_1, \dots, Y_n | \theta, \sigma^2 \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(\theta, \sigma^2)$

By conjugacy, the posterior is $\mathcal{N}(\mu_n, \tau_n^2)$ with

$$\mu_n = \frac{\frac{n}{\sigma^2}}{\frac{n}{\sigma^2} + \frac{1}{\tau^2}} \bar{y} + \frac{\frac{1}{\tau^2}}{\frac{n}{\sigma^2} + \frac{1}{\tau^2}} \mu, \quad \tau_n^2 = \frac{1}{\frac{n}{\sigma^2} + \frac{1}{\tau^2}}, \quad \text{where } \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i.$$

With $\sigma^2 = 1$, $\tau^2 = 10$, $\mu = 5$, $n = 5$, and $y = (9.37, 10.18, 9.16, 11.60, 10.33)$ we have $\bar{y} = 10.128$, hence

$$\mu_n \approx 10.0275 \text{ (rounded 10.03)}, \quad \tau_n^2 = \frac{1}{5 + 0.1} \approx 0.1961 \text{ (rounded 0.20)}.$$

Thus $p(\theta | y) = \mathcal{N}(10.03, 0.20)$ to two decimals.

Sampling this posterior via Metropolis. Use a symmetric random-walk proposal $\theta' \sim \mathcal{N}(\theta^{(s)}, \delta^2)$, so the acceptance probability is

$$A = \min\{1, r\}, \quad r = \frac{p(\theta' | y)}{p(\theta^{(s)} | y)} = \frac{\prod_{i=1}^n \mathcal{N}(y_i | \theta', \sigma^2) \mathcal{N}(\theta' | \mu, \tau^2)}{\prod_{i=1}^n \mathcal{N}(y_i | \theta^{(s)}, \sigma^2) \mathcal{N}(\theta^{(s)} | \mu, \tau^2)}.$$

For numerical stability, work on the log scale. Using the identity

$$\sum_{i=1}^n (y_i - \theta)^2 = \sum_{i=1}^n (y_i - \bar{y})^2 + n(\theta - \bar{y})^2,$$

the log-acceptance ratio simplifies to a *quadratic* in the proposals:

$$\log r = -\frac{n}{2\sigma^2} \left[(\theta' - \bar{y})^2 - (\theta^{(s)} - \bar{y})^2 \right] - \frac{1}{2\tau^2} \left[(\theta' - \mu)^2 - (\theta^{(s)} - \mu)^2 \right].$$

This form depends only on the sufficient statistic $\bar{y}$ (no need to rescan the entire data vector at every step).

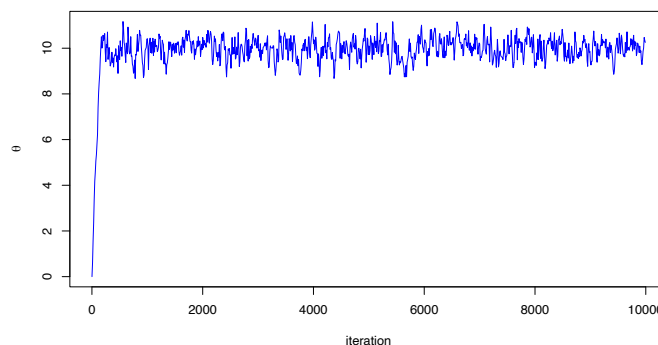
R code. RW-Metropolis with proposal distribution $\theta' \sim \mathcal{N}(\theta^{(s)}, \delta^2)$, with proposal variance $\delta^2 = 2$, and with starting point $\theta^{(0)} = 0$.

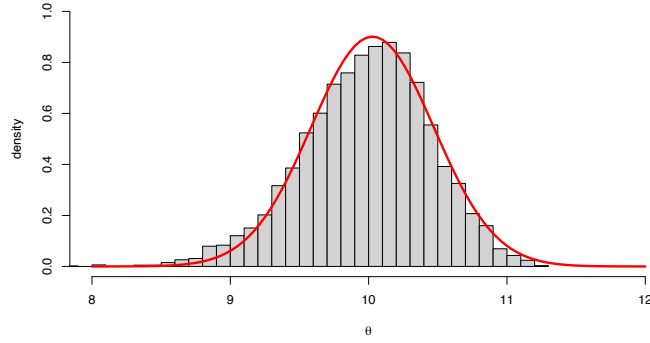
```
s2 <- 1
t2 <- 10 ; mu <- 5
y <- c(9.37, 10.18, 9.16, 11.60, 10.33)
n <- length(y) ; ybar <- mean(y)

S <- 10000
theta <- 0 # start value
delta2 <- 2 # proposal variance
sprop <- sqrt(delta2)
THETA <- numeric(S)
acc <- 0L

for (s in 1:S) {
  theta_p <- rnorm(1, mean = theta, sd = sprop)
  log_r <- -(n/(2*s2))*((theta_p - ybar)^2 - (theta - ybar)^2) -
    (1/(2*t2))*((theta_p - mu)^2 - (theta - mu)^2)
  if (log(runif(1)) < log_r) { theta <- theta_p ; acc <- acc + 1L }
  THETA[s] <- theta
}
acc_rate <- acc / S
```

The figures below show (i) a trace plot of $\{\theta^{(s)}\}$ and (ii) a histogram of the draws overlaid with the analytic posterior $\mathcal{N}(10.03, 0.20)$.





Tuning the proposal scale. The correlation among successive $\theta^{(s)}$ values (and hence the effective sample size) depends on δ^2 :

- Too small $\delta^2 \Rightarrow$ very high acceptance but small steps and slow exploration.
- Too large $\delta^2 \Rightarrow$ many rejections and long “sticky” stretches.
- A moderate δ^2 balances move size and acceptance.

Even though we can sample directly from the conjugate posterior here, MH illustrates how to construct valid samplers when closed forms are unavailable.

Example 2. Estimating a correlation with Metropolis

We observe pairs $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ assumed conditionally iid given the parameter of the joint sampling model of (X, Y) . We are interested in estimating the correlation ρ between variables X and Y in the sampling model.

Prior. The correlation parameter ρ lies between -1 and 1 . We assume a uniform prior distribution $\rho \sim \text{Unif}(-1, 1)$, with density $p_\rho(\rho) = 1/2$ for $\rho \in (-1, 1)$ and $p_\rho(\rho) = 0$ otherwise.

Sampling model. Samples are conditionally iid from the following bivariate normal with mean zero and unit variances,

$$\begin{bmatrix} X \\ Y \end{bmatrix} \bigg| \rho \sim \mathcal{N} \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix} \right), \quad \rho \in (-1, 1).$$

(If your data are not standardized, center and scale first or extend the model to include means/variances.)

That is, the density of $\mathcal{D}$ given ρ is given by

$$p_{\mathcal{D}|\rho}((x_1, y_1), \dots, (x_n, y_n) | \rho)$$

$$\begin{aligned}
&= \prod_{i=1}^n \frac{1}{2\pi\sqrt{1-\rho^2}} \exp\left(-\frac{x_i^2 + y_i^2 - 2\rho x_i y_i}{2(1-\rho^2)}\right) \\
&= \left(\frac{1}{4\pi^2(1-\rho^2)}\right)^{n/2} \exp\left(-\frac{1}{2(1-\rho^2)}(S_{xx} + S_{yy} - 2\rho S_{xy})\right),
\end{aligned}$$

where $S_{xx} = \sum_{i=1}^n x_i^2$, $S_{yy} = \sum_{i=1}^n y_i^2$, and $S_{xy} = \sum_{i=1}^n x_i y_i$.

Then

$$p(\mathcal{D} \mid \rho) \propto (1 - \rho^2)^{-n/2} \exp\left\{-\frac{S_{xx} + S_{yy} - 2\rho S_{xy}}{2(1 - \rho^2)}\right\},$$

Posterior. With the uniform prior

$$p(\rho \mid \mathcal{D}) \propto \tilde{p}(\rho) := (1 - \rho^2)^{-n/2} \exp\left\{-\frac{S_{xx} + S_{yy} - 2\rho S_{xy}}{2(1 - \rho^2)}\right\}, \quad |\rho| < 1.$$

The log-unnormalized posterior is

$$\log \tilde{p}(\rho) = -\frac{n}{2} \log(1 - \rho^2) - \frac{S_{xx} + S_{yy} - 2\rho S_{xy}}{2(1 - \rho^2)}.$$

Goal. Compute posterior summaries for ρ (e.g., mean and a 95% credible interval) via Metropolis.

Remark. Respect the support: proposals outside $(-1, 1)$ are automatically rejected.

R code (data generation for illustration).

```

rho_star <- 0.9
n <- 100
Z1 <- rnorm(n); Z2 <- rnorm(n)
Y <- rho_star*Z1 + sqrt(1 - rho_star^2)*Z2
X <- cbind(Z1, Y) # n x 2 matrix
# sufficient statistics:
Sxx <- sum(X[,1]^2); Syy <- sum(X[,2]^2); Sxy <- sum(X[,1]*X[,2])

```

R code (RW-Metropolis directly on ρ).

```

logpost_rho <- function(rho, n, Sxx, Syy, Sxy){
  if (abs(rho) >= 1) return(-Inf)
  -0.5*n*log1p(-rho^2) - 0.5*(Sxx + Syy - 2*rho*Sxy)/(1 - rho^2)
}

Niter <- 30000
rho <- 0 # start value

```

```

sigma <- 0.01                                # proposal sd
draws <- numeric(Niter)
acc <- 0L

for (i in 1:Niter) {
  rho_prop <- rho + sigma*rnorm(1)
  log_r <- logpost_rho(rho_prop, n, Sxx, Syy, Sxy) -
    logpost_rho(rho, n, Sxx, Syy, Sxy)
  if (log(runif(1)) < log_r) { rho <- rho_prop; acc <- acc + 1L }
  draws[i] <- rho
}
acc_rate <- acc / Niter

```

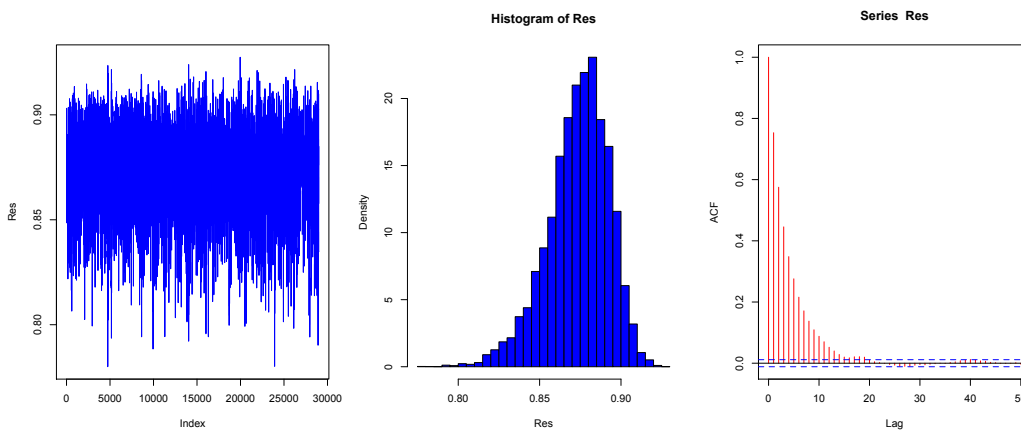
Numerical stability: use $\log 1p(-\rho^2)$ for $\log(1 - \rho^2)$ near the boundaries.

Posterior summaries and diagnostics. Discard a burn-in (e.g., first 5,000), then compute

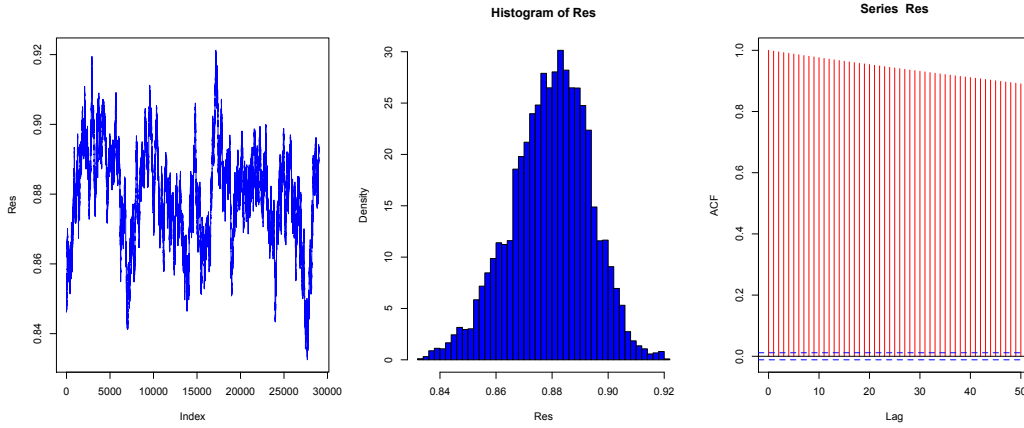
$$\widehat{\mathbb{E}[\rho \mid \mathcal{D}]} \approx \frac{1}{S^*} \sum_{s=1}^{S^*} \rho^{(s)}, \quad 95\% \text{ CrI} \approx [\rho^{(s)}]_{2.5\%, 97.5\%}.$$

Inspect *trace* and *ACF* plots; report the effective sample size S_{eff} and Monte Carlo SE. Tuning the proposal scale (σ or s_z) is important: too small $\Rightarrow$ slow movement; too large $\Rightarrow$ many rejections. In 1D, acceptance rates around 0.4–0.5 are typically reasonable.

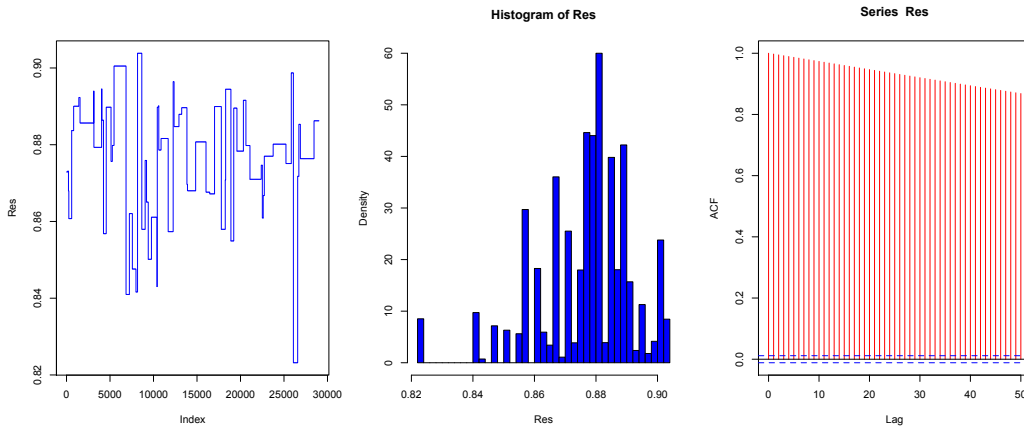
Trace, histogram, and ACF. With a well-tuned proposal (e.g., $\sigma = 0.01$ below), the chain traverses the space rapidly and the ACF decays quickly:



Too small a step size ($\sigma = 0.001$) yields highly persistent draws (high autocorrelation):



Too large ($\sigma = 10$) causes frequent rejections and “sticky” traces:



Question: How do we know we have a well behaved MCMC algorithm?

There is no clear answer to that question. But we can easily sort out overtly bad MCMC algorithms. We want MCMC samplers that “walk around” or “move around” quickly. To get some idea on how fast the Markov chain is moving around we look at the trace plot, and the auto-correlation function plot. Ideally the auto-correlation function should look like the correlation function of iid random variables. The auto-correlation function gives (for increasing values of k) the sample version of the auto-correlation $\text{Corr}(z^{(s)}, z^{(s+k)})$. For $k = 0$, $\text{corr}(z^{(s)}, z^{(s+k)}) = 1$. In the iid case, $\text{corr}(z^{(s)}, z^{(s+k)}) = 0$ for $k \geq 1$. Thus the goal is to have $\text{corr}(z^{(s)}, z^{(s+k)})$ as small as possible for $k \geq 1$.

It is also very important to realize that the choice of σ plays an important role. And a good value of σ depends on the target distribution p .

4.2 Metropolis–Hastings Algorithm (asymmetric proposal)

With an *asymmetric* proposal $q(z' | z) \neq q(z | z')$, include the Hastings correction:

$$A(z' | z) = \min \left\{ 1, \frac{\tilde{p}(z') q(z | z')}{\tilde{p}(z) q(z' | z)} \right\}.$$

Algorithm 2 Metropolis–Hastings Algorithm

```

Initialize:  $z^{(0)}$ 
for  $s = 0, 1, \dots$  do
   $z \leftarrow z^{(s)}$ 
  Propose  $z' \sim q(z' | z)$ 
  Compute  $A = \min \left\{ 1, \frac{\tilde{p}(z') q(z | z')}{\tilde{p}(z) q(z' | z)} \right\}$ 
  Draw  $U \sim \text{Unif}(0, 1)$ 
  if  $U \leq A$  then
     $z^{(s+1)} \leftarrow z'$  ▷ accept
  else
     $z^{(s+1)} \leftarrow z$  ▷ reject
  end if
end for

```

Remarks.

- The Hastings factor compensates for proposals that themselves (rather than just the target distribution) prefer certain directions or regions, restoring the correct long-run frequency under p .
- Clearly, if q is symmetric, Metropolis–Hastings rule reduces to the Metropolis rule.
- Notice that again we work with $\tilde{p}$, and never need to worry about the normalizing constant C .
- A more general form of Metropolis–Hastings is that the proposal distribution depends on iteration number s (with diminishing adaptation). Special case is to use *mixtures* or *cycles* of proposals (e.g., different scales or blocks updated in turn). For example, we can have a different proposal distribution for odd values of s compared to even values of s . We will later see that this is instrumental in Gibbs samplers, which is a special case of Metropolis–Hastings algorithm. Using a fixed finite set of kernels each iteration—either at random (mixture) or in a fixed order (composition)—still preserves p .

4.2.1 Why Metropolis–Hastings Algorithm Works?

Let T denote the one-step transition function of MH chain. On a *discrete* space,

$$T(z, z') = \begin{cases} q(z' | z) A(z' | z), & z' \neq z, \\ q(z | z) + \sum_{z' \neq z} q(z' | z) [1 - A(z' | z)], & z' = z, \end{cases}$$

and on a *continuous* state space,

$$T(z, z') = \begin{cases} q(z' | z) A(z' | z), & z' \neq z, \\ 1 - \int_{\mathcal{Z} \setminus \{z\}} q(u | z) A(u | z) du, & z' = z. \end{cases}$$

Recall that *detailed balance* of p with respect to T requires

$$p(z) T(z, z') = p(z') T(z', z) \quad \text{for all } z \text{ and } z'.$$

Theorem 4.1. The target p satisfies detailed balance with respect to the MH transition function T ; hence p is the stationary distribution of the Markov chain.

Proof. For $z' = z$, the equality

$$p(z) T(z, z') = p(z') T(z', z)$$

trivially holds.

For $z \neq z'$,

$$p(z) T(z, z') = C \tilde{p}(z) q(z' | z) \min \left\{ 1, \frac{\tilde{p}(z') q(z | z')}{\tilde{p}(z) q(z' | z)} \right\}.$$

Using the identity $a \min\{1, b/a\} = b \min\{1, a/b\}$ for $a, b > 0$, the right-hand side equals

$$C \tilde{p}(z') q(z | z') \min \left\{ 1, \frac{\tilde{p}(z) q(z' | z)}{\tilde{p}(z') q(z | z')} \right\} = p(z') T(z', z).$$

Thus detailed balance holds and p is invariant. □

Remark (ergodicity). If, in addition, the chain is irreducible, aperiodic, and recurrent, then by Theorem 2.2 p is the *unique* stationary distribution and time averages converge to $\mathbb{E}_p[f(Z)]$. For example, if p is continuous, then using a normal proposal distribution centered around the current value guarantees that $\mathbb{P}(z^{(s+1)} \in A \mid z^{(s)} = z) > 0$ for every z , s , and set A such that $p(A) > 0$.

5 Gibbs Sampler

Suppose the state is two-dimensional, $Z = (U, V)$. In a general Metropolis–Hastings (MH) scheme the proposal can change both coordinates at once. A common special case is *coordinate-wise MH*, which updates one coordinate at a time while holding the other fixed. Allowing the proposal to depend on the current iteration s , a single sweep takes the form:

1. **Update U (given $V = v^{(s)}$):**

$$\begin{aligned} u' &\sim q_u(u' \mid u^{(s)}, v^{(s)}), \\ \alpha_u &= \min \left\{ 1, \frac{\tilde{p}(u', v^{(s)}) q_u(u^{(s)} \mid u', v^{(s)})}{\tilde{p}(u^{(s)}, v^{(s)}) q_u(u' \mid u^{(s)}, v^{(s)})} \right\}, \\ u^{(s+1)} &= \begin{cases} u', & \text{with prob. } \alpha_u, \\ u^{(s)}, & \text{otherwise.} \end{cases} \end{aligned}$$

2. **Update V (given $U = u^{(s+1)}$):**

$$\begin{aligned} v' &\sim q_v(v' \mid u^{(s+1)}, v^{(s)}), \\ \alpha_v &= \min \left\{ 1, \frac{\tilde{p}(u^{(s+1)}, v') q_v(v^{(s)} \mid u^{(s+1)}, v')}{\tilde{p}(u^{(s+1)}, v^{(s)}) q_v(v' \mid u^{(s+1)}, v^{(s)})} \right\}, \\ v^{(s+1)} &= \begin{cases} v', & \text{with prob. } \alpha_v, \\ v^{(s)}, & \text{otherwise.} \end{cases} \end{aligned}$$

Now specialize further by choosing the proposals to be the *full conditionals*:

- $q_u(u' \mid u, v) = p(u' \mid v)$: the full conditional distribution of u' given $V = v$
- $q_v(v' \mid u, v) = p(v' \mid u)$: the full conditional distribution of v' given $U = u$

This yields the *Gibbs sampler*. The MH ratio for the U -update simplifies to

$$\frac{\tilde{p}(u', v^{(s)}) q_u(u^{(s)} \mid u', v^{(s)})}{\tilde{p}(u^{(s)}, v^{(s)}) q_u(u' \mid u^{(s)}, v^{(s)})} = \frac{p(u', v^{(s)}) p(u^{(s)} \mid v^{(s)})}{p(u^{(s)}, v^{(s)}) p(u' \mid v^{(s)})} = \frac{p(u' \mid v^{(s)}) p(v^{(s)}) p(u^{(s)} \mid v^{(s)})}{p(u^{(s)} \mid v^{(s)}) p(v^{(s)}) p(u' \mid v^{(s)})} = 1,$$

and analogously for the V -update. Hence *every* Gibbs proposal is accepted.

For M dimensional state, the Gibbs sampler would be as follows:

The density $p(z_i \mid z_{-i})$ is the *full conditional* for Z_i . A random-scan variant updates a randomly chosen coordinate each step; both preserve the target.

Remark. A 100% acceptance rate does not imply rapid convergence. Gibbs may mix slowly when coordinates are strongly correlated (single-coordinate moves explore along narrow ridges).

Algorithm 3 Gibbs Sampler

```

Initialize  $z^{(0)} = (z_1^{(0)}, \dots, z_M^{(0)})$ 
for  $s = 0, 1, 2, \dots$  do
  Sample  $z_1^{(s+1)} \sim p(z_1 \mid z_2^{(s)}, \dots, z_M^{(s)})$ 
  Sample  $z_2^{(s+1)} \sim p(z_2 \mid z_1^{(s+1)}, z_3^{(s)}, \dots, z_M^{(s)})$ 
   $\vdots$ 
  Sample  $z_j^{(s+1)} \sim p(z_j \mid z_1^{(s+1)}, \dots, z_{j-1}^{(s+1)}, z_{j+1}^{(s)}, \dots, z_M^{(s)})$ 
   $\vdots$ 
  Sample  $z_M^{(s+1)} \sim p(z_M \mid z_1^{(s+1)}, \dots, z_{M-1}^{(s+1)})$ 
end for

```

5.1 Gibbs Sampling for the Normal Model

For many multi-parameter models the joint posterior distribution is non-standard and difficult to sample from directly. However, it is often the case that it is easy to sample from the full conditional distribution of each parameter. In such cases, posterior approximation can be made with the Gibbs sampler. Recall the normal model from Chapter 5. We showed that for the normal sampling model

$$Y_1, \dots, Y_n \mid \theta, \sigma^2 \stackrel{\text{iid}}{\sim} \mathcal{N}(\theta, \sigma^2),$$

the following normal-inverse-gamma prior distribution is the conjugate prior:

$$\sigma^2 \sim \text{IG}\left(\frac{\nu_0}{2}, \frac{\nu_0 \sigma_0^2}{2}\right) \quad \theta \mid \sigma^2 \sim \mathcal{N}(\mu_0, \tau_0^2),$$

Importantly, this result is contingent on choosing

$$\tau_0^2 = \frac{\sigma^2}{\kappa_0}.$$

Hence, we were able to easily sample from the joint posterior distribution by sampling

1. $\sigma^{2(s)}$ from posterior inverse-gamma distribution $p(\sigma^2 \mid y_1, \dots, y_n)$,
2. $\theta^{(s)}$ from posterior conditional normal distribution $p(\theta \mid \sigma^{2(s)}, y_1, \dots, y_n)$.

What if we do not want to assume $\tau_0^2 = \frac{\sigma^2}{\kappa_0}$? For example, if we wanted to have a-priori independence between the parameters, i.e., $p(\theta, \sigma^2) = p(\theta)p(\sigma^2)$? In this case, we will not have **full conjugacy** anymore. However, we can still do sampling from the joint posterior using Gibbs sampler.

Recall that

$$\text{If } \theta \mid \sigma^2 \sim \mathcal{N}(\mu_0, \tau_0^2), \quad \text{then} \quad \theta \mid \sigma^2, y_1, \dots, y_n \sim \mathcal{N}(\mu_n, \tau_n^2).$$

Also, (from the homework) recall that

$$\text{If } \sigma^2 \mid \theta \sim \text{IG} \left(\frac{\nu_0}{2}, \frac{\nu_0 \sigma_0^2}{2} \right), \quad \text{then} \quad \sigma^2 \mid \theta, y_1, \dots, y_n \sim \text{IG} \left(\frac{\nu_n}{2}, \frac{\nu_n \sigma_n^2}{2} \right),$$

where

$$\tau_n^2 = \left(\frac{n}{\sigma^2} + \frac{1}{\tau_0^2} \right)^{-1}, \quad \mu_n = \tau_n^2 \left(\frac{n\bar{y}}{\sigma^2} + \frac{\mu_0}{\tau_0^2} \right), \quad \nu_n = \nu_0 + n, \quad \sigma_n^2 = \frac{1}{\nu_n} \left(\nu_0 \sigma_0^2 + \sum_{i=1}^n (y_i - \theta)^2 \right).$$

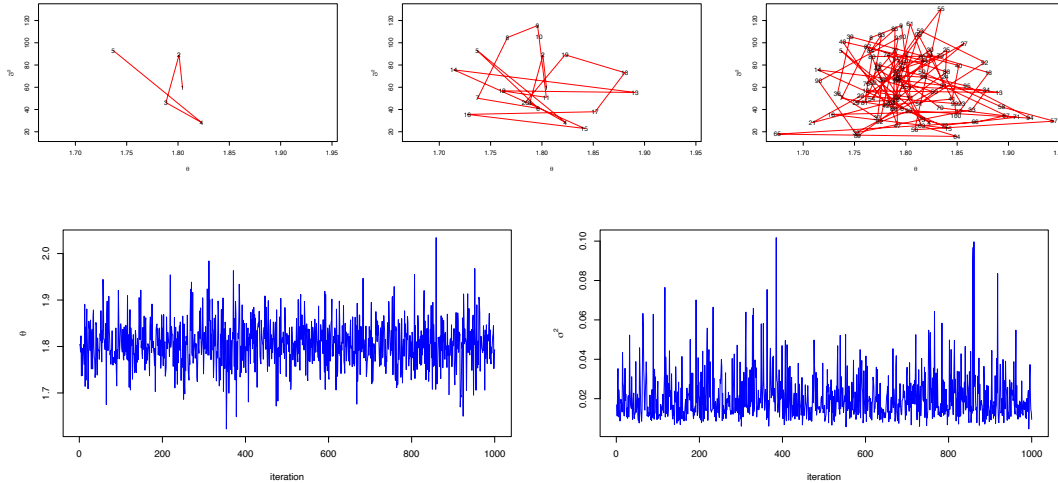
Such conditional conjugacy-type relationship is called **semi-conjugacy**. Specifically, normal distribution and IG distribution together are semi-conjugate distributions for the normal sampling model. This property enables us to easily sample directly from both $p(\theta \mid \sigma^2, y_1, \dots, y_n)$ and $p(\sigma^2 \mid \theta, y_1, \dots, y_n)$. Therefore, Gibbs sampling can be applied. Alternating draws from these two conditionals yields a valid Gibbs sampler targeting $p(\theta, \sigma^2 \mid y)$.

Definition 5.1 (semi-conjugate prior). For a sampling model $p(y \mid \theta, \sigma^2)$, a prior $p(\theta, \sigma^2)$ is called *semi-conjugate* if the full conditionals $p(\theta \mid \sigma^2, y)$ and $p(\sigma^2 \mid \theta, y)$ are in the same parametric families as $p(\theta \mid \sigma^2)$ and $p(\sigma^2 \mid \theta)$, respectively, for all y . It does *not* require the joint posterior to be from the same family as the joint prior.

Example.

Data: $\{n = 9, \bar{y} = 1.804, s^2 = 0.017\}$

Prior parameters: $\mu_0 = 1.9, \tau_0 = 0.95, \nu_0 = 1, \sigma_0^2 = 0.01$



5.2 Multivariate Normal Model

The approach above can be extended to the multivariate normal model. In Chapter 5, we saw that for the multivariate normal sampling model

$$\mathbf{Y}_1, \mathbf{Y}_2, \dots, \mathbf{Y}_n \mid \boldsymbol{\theta}, \boldsymbol{\Sigma} \stackrel{\text{iid}}{\sim} \mathcal{N}_p(\boldsymbol{\theta}, \boldsymbol{\Sigma}),$$

the following normal-inverse-Wishart prior distribution is the conjugate prior:

$$\boldsymbol{\theta} \mid \boldsymbol{\Sigma} \sim \mathcal{N}_p(\boldsymbol{\mu}_0, \boldsymbol{\Lambda}_0), \quad \boldsymbol{\Sigma} \sim \text{IW}_p(\nu_0, \boldsymbol{\Phi}_0).$$

Importantly, conjugacy hinges on the coupling

$$\boldsymbol{\Lambda}_0 = \boldsymbol{\Sigma} / \kappa_0,$$

i.e., the prior variance of the mean is proportional to $\boldsymbol{\Sigma}$.

We now relax this requirement by looking at full conditionals and taking a Gibbs sampling approach.

Suppose $\boldsymbol{\Sigma}$ has the inverse-Wishart prior $\text{IW}_p(\nu_0, \boldsymbol{\Phi}_0)$ with ν_0 degrees of freedom and **scale** matrix $\boldsymbol{\Phi}_0$. Recall from Chapter 5 that this implies that

$$p(\boldsymbol{\Sigma}) \propto |\boldsymbol{\Sigma}|^{-(\nu_0+p+1)/2} \exp\left\{-\frac{1}{2} \text{tr}(\boldsymbol{\Phi}_0 \boldsymbol{\Sigma}^{-1})\right\}.$$

Our sampling model satisfies

$$\begin{aligned} p(\mathbf{y}_1, \dots, \mathbf{y}_n \mid \boldsymbol{\theta}, \boldsymbol{\Sigma}) &\propto |\boldsymbol{\Sigma}|^{-\frac{n}{2}} \exp\left\{-\frac{1}{2} \sum_{i=1}^n (\mathbf{y}_i - \boldsymbol{\theta})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{y}_i - \boldsymbol{\theta})\right\} \\ &= |\boldsymbol{\Sigma}|^{-\frac{n}{2}} \exp\left\{-\frac{1}{2} \text{tr}(\boldsymbol{\Phi}_\theta \boldsymbol{\Sigma}^{-1})\right\}, \end{aligned}$$

where $\boldsymbol{\Phi}_\theta = \sum_{i=1}^n (\mathbf{y}_i - \boldsymbol{\theta})(\mathbf{y}_i - \boldsymbol{\theta})^\top$, and we used the result from matrix algebra that if $B = \sum_{i=1}^n b_i b_i^\top$, then $\sum_{i=1}^n b_i^\top M b_i = \text{tr}(BM)$. Note that $\boldsymbol{\Phi}_\theta$ is the sum of square residuals matrix for vectors $y_1, \dots, y_n$. Conditional on $\boldsymbol{\theta}$, $\frac{1}{n} \boldsymbol{\Phi}_\theta$ provides an unbiased estimate of the true covariance matrix.

Combining the prior and the sampling model, we have

$$\begin{aligned} p(\boldsymbol{\Sigma} \mid \mathbf{y}_1, \dots, \mathbf{y}_n, \boldsymbol{\theta}) &\propto p(\boldsymbol{\Sigma}) p(\mathbf{y}_1, \dots, \mathbf{y}_n \mid \boldsymbol{\theta}, \boldsymbol{\Sigma}) \\ &\propto \left[|\boldsymbol{\Sigma}|^{-(\nu_0+p+1)/2} \exp\left\{-\frac{1}{2} \text{tr}(\boldsymbol{\Phi}_0 \boldsymbol{\Sigma}^{-1})\right\} \right] \times \left[|\boldsymbol{\Sigma}|^{-\frac{n}{2}} \exp\left\{-\frac{1}{2} \text{tr}(\boldsymbol{\Phi}_\theta \boldsymbol{\Sigma}^{-1})\right\} \right] \\ &= |\boldsymbol{\Sigma}|^{-(\nu_0+n+p+1)/2} \exp\left\{-\frac{1}{2} \text{tr}([\boldsymbol{\Phi}_0 + \boldsymbol{\Phi}_\theta] \boldsymbol{\Sigma}^{-1})\right\}. \end{aligned}$$

Thus we have

$$\boldsymbol{\Sigma} \mid \mathbf{y}_1, \dots, \mathbf{y}_n, \boldsymbol{\theta} \sim \text{IW}_p([\nu_0 + n], [\boldsymbol{\Phi}_0 + \boldsymbol{\Phi}_\theta]).$$

Gibbs sampling of the mean and covariance. We showed that

$$\boldsymbol{\theta} \mid \mathbf{y}_1, \dots, \mathbf{y}_n, \boldsymbol{\Sigma} \sim \text{multivariate normal}(\boldsymbol{\mu}_n, \boldsymbol{\Lambda}_n),$$

$$\boldsymbol{\Sigma} \mid \mathbf{y}_1, \dots, \mathbf{y}_n, \boldsymbol{\theta} \sim \text{inverse-Wishart}(\nu_n, \boldsymbol{\Phi}_n),$$

where $\nu_n = \nu_0 + n$, $\boldsymbol{\Phi}_n = \boldsymbol{\Phi}_0 + \boldsymbol{\Phi}_\theta$ with $\boldsymbol{\Phi}_\theta = \sum_{i=1}^n (\mathbf{y}_i - \boldsymbol{\theta})(\mathbf{y}_i - \boldsymbol{\theta})^\top$, $\boldsymbol{\Lambda}_n = \left(\boldsymbol{\Lambda}_0^{-1} + n \boldsymbol{\Sigma}^{-1} \right)^{-1}$, and $\boldsymbol{\mu}_n = \boldsymbol{\Lambda}_n \left(\boldsymbol{\Lambda}_0^{-1} \boldsymbol{\mu}_0 + n \boldsymbol{\Sigma}^{-1} \bar{\mathbf{y}} \right)$.

These full conditional distributions can be used to construct a Gibbs sampler, providing us with an MCMC approximation to the joint posterior distribution $p(\boldsymbol{\theta}, \boldsymbol{\Sigma} \mid \mathbf{y}_1, \dots, \mathbf{y}_n)$. Given a starting value $\boldsymbol{\Sigma}^{(0)}$, the Gibbs sampler generates $\{\boldsymbol{\theta}^{(s+1)}, \boldsymbol{\Sigma}^{(s+1)}\}$ from $\{\boldsymbol{\theta}^{(s)}, \boldsymbol{\Sigma}^{(s)}\}$ via the following two steps:

1. Sample $\boldsymbol{\theta}^{(s+1)}$ from its full conditional distribution:
 - (a) compute $\boldsymbol{\mu}_n$ and $\boldsymbol{\Lambda}_n$ from $\mathbf{y}_1, \dots, \mathbf{y}_n$ and $\boldsymbol{\Sigma}^{(s)}$;
 - (b) sample $\boldsymbol{\theta}^{(s+1)} \sim \text{multivariate normal}(\boldsymbol{\mu}_n, \boldsymbol{\Lambda}_n)$.
2. Sample $\boldsymbol{\Sigma}^{(s+1)}$ from its full conditional distribution:
 - (a) compute $\boldsymbol{\Phi}_n$ from $\mathbf{y}_1, \dots, \mathbf{y}_n$ and $\boldsymbol{\theta}^{(s+1)}$;
 - (b) sample $\boldsymbol{\Sigma}^{(s+1)} \sim \text{inverse-Wishart}(\nu_n, \boldsymbol{\Phi}_n)$.

5.3 Bayesian Linear Regression via Gibbs

Consider fixed design matrix $X \in \mathbb{R}^{n \times p}$ and response $y \in \mathbb{R}^n$,

$$y \mid \beta, \sigma^2, X \sim \mathcal{N}_n(X\beta, \sigma^2 I_n).$$

Recall from Chapter 6 that in the Bayesian framework for linear regression with fixed design matrix X , inference proceeds by combining the likelihood function with a prior distribution to obtain the posterior:

$$p(\beta, \sigma^2 \mid y, X) \propto p(y \mid X, \beta, \sigma^2) p(\beta, \sigma^2 \mid X).$$

We showed that the conjugate prior for β (conditional on σ^2) is a multivariate normal distribution $\mathcal{N}(\beta_0, \Sigma_0)$. We focused on Zellner's g -prior in that chapter which uses the following specific choice of the hyper parameters of the prior distribution that reflects the geometry of the design matrix X :

$$\beta_0 = 0, \quad \Sigma_0 = g\sigma^2(X^\top X)^{-1},$$

for some positive value g , giving the g -prior:

$$\beta \mid \sigma^2, X \sim \mathcal{N}(0, g\sigma^2(X^\top X)^{-1}).$$

One of the main desirabilities of this choice was that it lead to normal-inverse-gamma being the conjugate prior. We now consider a general choice for (β_0, Σ_0) . Similar to the previous subsections, we first prove semi-conjugacy.

Claim. The semi-conjugate prior distribution for σ^2 is an inverse-gamma distribution

Proof. If $\sigma^2 \sim \text{IG}(\nu_0/2, \nu_0\sigma_0^2/2)$, then

$$\begin{aligned} p\left(\frac{1}{\sigma^2} \mid y, X, \beta\right) &\propto p\left(\frac{1}{\sigma^2}\right)p(y \mid X, \beta, \frac{1}{\sigma^2}) \\ &\propto \left[\left(\frac{1}{\sigma^2}\right)^{\nu_0/2-1} \exp\left(-\left(\frac{1}{\sigma^2}\right)\nu_0\sigma_0^2/2\right)\right] \times \left[\left(\frac{1}{\sigma^2}\right)^{n/2} \exp\left(-\left(\frac{1}{\sigma^2}\right)\text{SSR}(\beta)/2\right)\right] \\ &= \left(\frac{1}{\sigma^2}\right)^{(\nu_0+n)/2-1} \exp\left(-\left(\frac{1}{\sigma^2}\right)[\nu_0\sigma_0^2 + \text{SSR}(\beta)]/2\right), \end{aligned}$$

where $\text{SSR}(\beta) = (y - X\beta)^\top (y - X\beta)$. This has the same shape as a gamma distribution. Therefore

$$\sigma^2 \mid y, X, \beta \sim \text{IG}([\nu_0 + n]/2, [\nu_0\sigma_0^2 + \text{SSR}(\beta)]/2)$$

□

Full conditionals.

$$\beta \mid \sigma^2, y, X \sim \mathcal{N}_p(m, V), \quad V = \left(\Sigma_0^{-1} + \frac{1}{\sigma^2} X^\top X\right)^{-1}, m = V\left(\Sigma_0^{-1}\beta_0 + \frac{1}{\sigma^2} X^\top y\right);$$

$$\sigma^2 \mid \beta, y, X \sim \text{IG}\left(\frac{\nu_0 + n}{2}, \frac{\nu_0\sigma_0^2 + \text{SSR}(\beta)}{2}\right).$$

Now we can construct a Gibbs sampler to sample from the joint posterior distribution $p(\sigma^2, \beta \mid y, X)$:

Gibbs sampler. At iteration s :

1. Sample $\beta^{(s+1)} \sim \mathcal{N}_p(m^{(s)}, V^{(s)})$
with $V^{(s)} = (\Sigma_0^{-1} + X^\top X/\sigma^{2(s)})^{-1}$ and $m^{(s)} = V^{(s)}(\Sigma_0^{-1}\beta_0 + X^\top y/\sigma^{2(s)})$.
2. Sample $\sigma^{2(s+1)} \sim \text{IG}((\nu_0 + n)/2, (\nu_0\sigma_0^2 + \text{SSR}(\beta^{(s+1)}))/2)$.

5.4 Metropolis-within-Gibbs (MwG)

In Gibbs sampling we draw each coordinate from its *full conditional* $p(z_i \mid z_{-i})$. When a full conditional is not available in a tractable form or is hard to sample from directly, we can use a local Metropolis-Hastings (MH) update that *targets* the same full conditional in the step where we need to sample from the full conditional. This hybrid scheme is called **Metropolis-within-Gibbs** (MwG).

Formally, to sample z'_i from $p(z'_i \mid z_1^{(s+1)}, \dots, z_{i-1}^{(s+1)}, z_{i+1}^{(s)}, \dots, z_M^{(s)})$,

One MwG update of coordinate i . At iteration s , let z_{-i} denote all coordinates except i , using the most recent values (i.e., $z_1^{(s+1)}, \dots, z_{i-1}^{(s+1)}, z_{i+1}^{(s)}, \dots, z_M^{(s)}$). To update z_i :

1. **Propose** $z'_i \sim q_i(z'_i \mid z_i^{(s)}, z_{-i})$ (could be chosen to be $q(z'_i \mid z_i^{(s)})$).
2. **Accept** with probability

$$\begin{aligned} \alpha_i &= \min \left\{ 1, \frac{p(z'_i \mid z_{-i}) q_i(z_i^{(s)} \mid z'_i, z_{-i})}{p(z_i^{(s)} \mid z_{-i}) q_i(z'_i \mid z_i^{(s)}, z_{-i})} \right\} \\ &= \min \left\{ 1, \frac{\tilde{p}(z'_i, z_{-i}) q_i(z_i^{(s)} \mid z'_i, z_{-i})}{\tilde{p}(z_i^{(s)}, z_{-i}) q_i(z'_i \mid z_i^{(s)}, z_{-i})} \right\}, \end{aligned}$$

where $\tilde{p}$ is the unnormalized joint density (the normalizing constant cancels).

3. **Set** $z_i^{(s+1)} \leftarrow z'_i$ with probability α_i ; otherwise $z_i^{(s+1)} \leftarrow z_i^{(s)}$.

Algorithm 4 Metropolis-within-Gibbs

```

Initialize  $z^{(0)} = (z_1^{(0)}, \dots, z_M^{(0)})$ 
for  $s = 0, 1, 2, \dots$  do
  for  $i = 1, \dots, M$  do ▷ update each coordinate in turn
    Let  $z_{-i} \leftarrow (z_1^{(s+1)}, \dots, z_{i-1}^{(s+1)}, z_{i+1}^{(s)}, \dots, z_M^{(s)})$ 
    Propose  $z'_i \sim q_i(\cdot \mid z_i^{(s)}, z_{-i})$ 
     $\alpha_i \leftarrow \min \left\{ 1, \frac{\tilde{p}(z'_i, z_{-i}) q_i(z_i^{(s)} \mid z'_i, z_{-i})}{\tilde{p}(z_i^{(s)}, z_{-i}) q_i(z'_i \mid z_i^{(s)}, z_{-i})} \right\}$ 
    Draw  $U \sim \text{Unif}(0, 1)$ 
    if  $U \leq \alpha_i$  then
       $z_i^{(s+1)} \leftarrow z'_i$ 
    else
       $z_i^{(s+1)} \leftarrow z_i^{(s)}$ 
    end if
  end for
end for

```

Remark (Multiple MH substeps). You may perform $L_i > 1$ MH moves for coordinate i before proceeding to $i+1$; this still preserves p .

6 Hamiltonian Monte Carlo (HMC)

Random-walk Metropolis proposals tend to take many small steps: enlarging the step size usually increases the rejection rate, so the chain moves slowly in high dimensions. *Hamiltonian Monte Carlo (HMC)* overcomes this by using *gradient information* to simulate approximate *Hamiltonian dynamics* that carry the chain along contours of high probability,

allowing *large, distant moves with high acceptance*. HMC is one of the most effective MCMC methods for medium- to high-dimensional continuous targets.

This method builds on Hamiltonian mechanics from the field of physics.

6.1 Hamiltonian Mechanics (brief review)

Consider a particle rolling around a landscape in $\mathbb{R}^d$. We can characterize the motion of the particle in terms of

- its position $\theta \in \mathbb{R}^d$, and
- its momentum $\nu \in \mathbb{R}^d$

Phase space: Set of all possible values for (θ, ν) .

Let

- $\mathcal{E}(\theta)$: Potential energy of the particle (due to gravity pull),
- $\mathcal{K}(\nu)$: Kinetic energy of the particle (due to motion),
- $\mathcal{H}(\theta, \nu) = \mathcal{E}(\theta) + \mathcal{K}(\nu)$: Total energy of the particle.

$\mathcal{H}(\theta, \nu)$ is called the *Hamiltonian function* for the point (θ, ν) in the phase space.

Trajectories in phase space that have a constant energy are called *stable orbits*.

Hamilton's equations are continuous time differential equations that describe the trajectory $(\theta(t), \nu(t))$ of a particle within an energy level set.

$$\begin{aligned}\frac{d\theta}{dt} &= \frac{\partial \mathcal{H}}{\partial \nu} = \frac{\partial \mathcal{K}}{\partial \nu}, \\ \frac{d\nu}{dt} &= -\frac{\partial \mathcal{H}}{\partial \theta} = -\frac{\partial \mathcal{E}}{\partial \theta}.\end{aligned}$$

These dynamics conserve energy: $\frac{d}{dt}\mathcal{H}(\theta(t), \nu(t)) = 0$. To see why energy is conserved, note that

$$\begin{aligned}\frac{d\mathcal{H}}{dt} &= \sum_{i=1}^d \left[\frac{\partial \mathcal{H}}{\partial \theta_i} \frac{d\theta_i}{dt} + \frac{\partial \mathcal{H}}{\partial \nu_i} \frac{d\nu_i}{dt} \right] \\ &= \sum_{i=1}^d \left[\frac{\partial \mathcal{H}}{\partial \theta_i} \frac{\partial \mathcal{H}}{\partial \nu_i} - \frac{\partial \mathcal{H}}{\partial \nu_i} \frac{\partial \mathcal{H}}{\partial \theta_i} \right] \\ &= 0.\end{aligned}$$

They also preserve *phase-space volume*:

Liouville's Theorem: Hamiltonian dynamical systems preserve volume in phase space: If we consider a region within the phase space, as this region evolves under the equations of Hamiltonian dynamics, its shape may change but its volume will not.

Question: How can we simulate Hamilton's equations in discrete time?

Leapfrog integrator (discrete simulation). To simulate the ODEs numerically, HMC uses the *leapfrog* (a.k.a. Störmer–Verlet) integrator with step size $\eta > 0$: We first perform a half update of the momentum, then a full update of the position, and then finally another half update of the momentum, by a small amount, known as the *step size* η :

$$\begin{aligned}\nu_{t+1/2} &= \nu_t - \frac{\eta}{2} \frac{\partial \mathcal{E}(\theta_t)}{\partial \theta} \\ \theta_{t+1} &= \theta_t + \eta \frac{\partial \mathcal{K}(\nu_{t+1/2})}{\partial \nu} \\ \nu_{t+1} &= \nu_{t+1/2} - \frac{\eta}{2} \frac{\partial \mathcal{E}(\theta_{t+1})}{\partial \theta}\end{aligned}$$

For multiple steps, this is equivalent to a half-step momentum update at the beginning and end, with alternating full updates of θ and ν in between. Leapfrog is *time-reversible* and *volume-preserving*, and has small energy error (global error $O(\eta^2)$).

6.2 From Mechanics to Sampling

Let our target be $p(\theta) \propto \tilde{p}(\theta)$. Choose the potential energy

$$\mathcal{E}(\theta) = -\log \tilde{p}(\theta).$$

This makes sense because particles prefer to be in states with lower energy and the formula is telling us that the probability of states is decreasing with respect to the energy level of states.

Choose a quadratic kinetic energy

$$\mathcal{K}(\nu) = \frac{1}{2} \nu^\top \Sigma^{-1} \nu,$$

where $\Sigma \succ 0$ is a positive definite matrix called inverse mass matrix.

Note that this is the standard kinetic energy formula that you are familiar with from high school.

Then the *joint* density on phase space is

$$\begin{aligned}p(\theta, \nu) &= \frac{1}{Z} \exp(-\mathcal{H}(\theta, \nu)) \\ &= \frac{1}{Z} \exp\left(-\mathcal{E}(\theta) - \frac{1}{2} \nu^\top \Sigma^{-1} \nu\right) \\ &= \frac{1}{Z_\theta} \tilde{p}(\theta) \frac{1}{Z_\nu} \exp\left(-\frac{1}{2} \nu^\top \Sigma^{-1} \nu\right).\end{aligned}$$

Therefore, the marginal in ν is $\nu \sim \mathcal{N}(0, \Sigma)$, and the marginal in θ is exactly the target $p(\theta)$:

$$p(\theta) = \int p(\theta, \nu) d\nu = \frac{1}{Z_\theta} \exp(-\mathcal{E}(\theta)) \int \frac{1}{Z_\nu} \exp\left(-\frac{1}{2}\nu^\top \Sigma^{-1}\nu\right) d\nu = \frac{1}{Z_\theta} \exp(-\mathcal{E}(\theta)).$$

Because exact Hamiltonian flow preserves volume and energy, it follows that the Hamiltonian dynamics will leave $p(\theta, \nu)$ invariant. This can be seen by considering a small region of phase space over which $\mathcal{H}$ is approximately constant. If we follow the evolution of the Hamiltonian equations for a finite time, then the volume of this region will remain unchanged as will the value of $\mathcal{H}$ in this region, and hence the probability density, which is a function only of $\mathcal{H}$, will also be unchanged. This can be used to design a Markov chain whose *stationary distribution* is exactly $p(\theta, \nu)$, and hence the desired $p(\theta)$ marginal.

Gradients and updates for quadratic kinetic energy. With $\mathcal{K}(\nu) = \frac{1}{2}\nu^\top \Sigma^{-1}\nu$ we have $\nabla_\nu \mathcal{K}(\nu) = \Sigma^{-1}\nu$, so the leapfrog position update is $\theta_{t+1} = \theta_t + \eta \Sigma^{-1}\nu_{t+\frac{1}{2}}$.

6.3 The HMC Algorithm

In practice we simulate the dynamics approximately with leapfrog and correct the small discretization error by a Metropolis step.

Algorithm 5 Hamiltonian Monte Carlo (one iteration)

Input: current position θ_{cur} ; step size η ; number of steps L ; inverse mass matrix Σ .

Sample momentum: $\nu_0 \sim \mathcal{N}(0, \Sigma)$.

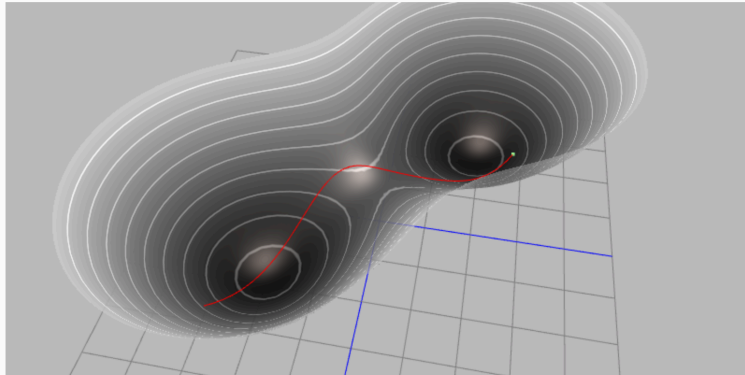
Integrate (leapfrog): starting at $(\theta_0, \nu_0) = (\theta_{\text{cur}}, \nu_0)$, apply L leapfrog steps in the phase space to obtain the proposed state (θ_L, ν_L) .

Accept/Reject:

$$\alpha = \min \left\{ 1, \frac{p(\theta_L, \nu_L)}{p(\theta_0, \nu_0)} \right\} = \min \{ 1, \exp(-\mathcal{H}(\theta_L, \nu_L) + \mathcal{H}(\theta_0, \nu_0)) \}.$$

With probability α set $\theta_{\text{new}} = \theta_L$; otherwise $\theta_{\text{new}} = \theta_{\text{cur}}$.

See the figure below for an example.



Remarks.

- In the Metropolis-Hastings acceptance probability design, the transition probabilities cancel since the proposal is reversible.
- The *acceptance probability* in HMC does not depend on an explicit proposal density, because the leapfrog map is deterministic and volume-preserving; the Metropolis ratio reduces to $\exp(-\Delta\mathcal{H})$ as shown above. The combination of (approximate) energy conservation, reversibility, and volume preservation yields a high-acceptance, long-distance proposal mechanism.
- **Why the acceptance ratio?** If leapfrog were exact (simulating the Hamiltonian dynamics perfectly), then the value of $\mathcal{H}$ would be unchanged and every proposal would be accepted. Due to numerical errors, the value of $\mathcal{H}$ may sometimes decrease, and we would like the Metropolis correction to remove any bias due to this effect and ensure that the resulting samples are indeed drawn from the required distribution.
- **Why resample the momentum each iteration?** Hamiltonian flow preserves total energy $\mathcal{H}$. If ν were fixed, the trajectory would stay on a single energy level set and could not explore regions with $\mathcal{E}(\theta) > \mathcal{H}$. Resampling $\nu \sim \mathcal{N}(0, \Sigma)$ each iteration changes the energy and prevents the chain from getting trapped on a single orbit.
- **Stationarity in HMC: flow vs. refresh.**
 - (i) (*Hamiltonian move*) The exact Hamiltonian flow is volume-preserving and energy-conserving, hence leaves joint distribution p invariant. In practice, a symplectic, volume-preserving, reversible integrator (e.g. leapfrog) combined with a Metropolis-Hastings correction restores exact invariance.
 - (ii) (*Momentum refresh*) Replacing v by an independent draw $v' \sim \mathcal{N}(0, \Sigma) = p(v \mid \theta)$ is a Gibbs update and therefore leaves the joint distribution p invariant, even though the realized Hamiltonian $H(\theta, v)$ typically changes.

Because both sub-kernels leave the same joint $p(\theta, v)$ invariant, their composition does as well; hence the marginal $p(\theta)$ is the stationary distribution of the θ -chain.

Tuning, diagnostics, and practicalities

- **Step size η and number of steps L .** Small η gives high acceptance but short moves (unless L is large); large η yields longer moves but more rejections due to larger energy error. Typical targets for the mean acceptance rate are ≈ 0.6 – 0.9 .
- **Inverse mass matrix Σ .** Choosing Σ to approximate the posterior covariance.
- **Energy error and acceptance.** For leapfrog the energy error after L steps is $O(\eta^2 L)$, so $\alpha \approx \exp(-\Delta\mathcal{H})$ is close to 1 when η is well tuned.
- **Tree depth / trajectory length.** If the integrator often hits the maximum trajectory length, consider increasing L .

- **Standard MCMC checks.** Trace/ACF plot, ESS, MCSE, and posterior checks.

A gentle tutorial with nice demos of Hamiltonian Monte Carlo:

http://arogozhnikov.github.io/2016/12/19/markov_chain_monte_carlo.html

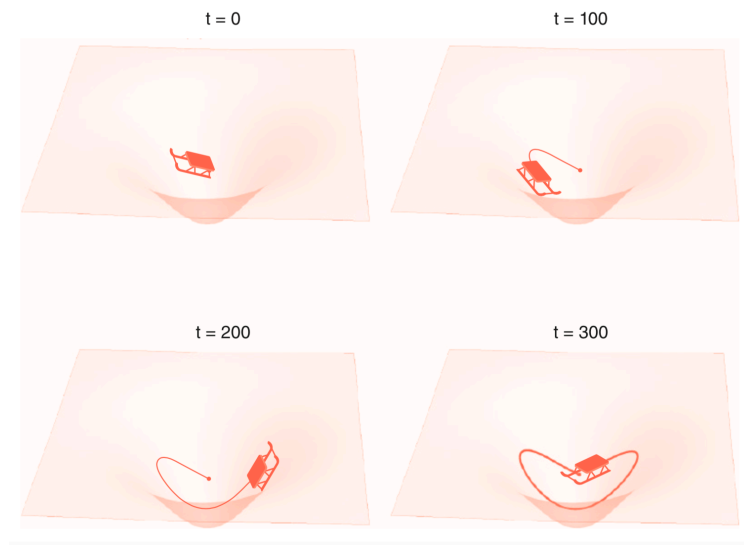
Can HMC be used for sampling *discrete* distributions? Not directly. Vanilla HMC is defined by Hamilton’s differential equations and the leapfrog integrator, which require a *continuously differentiable* log-density $\log \tilde{p}(\theta)$ and its gradient $\nabla_{\theta} \log \tilde{p}(\theta)$ on an open subset of $\mathbb{R}^d$. For discrete parameters (binary, counts, categories), the state space is not a differentiable manifold and the gradient is not defined, so the Hamiltonian dynamics—and hence standard HMC—do not apply.

What to do in models with discrete variables?

HMC-within-Gibbs for mixed discrete/continuous models. Update continuous blocks with HMC conditional on the current discrete values, and update the discrete blocks by Gibbs or Metropolis moves:

$$\theta^{(s+1)} \sim \text{HMC targeting } p(\theta \mid z^{(s)}, y), \quad z^{(s+1)} \sim \text{Gibbs/MH targeting } p(z \mid \theta^{(s+1)}, y).$$

This preserves the correct joint target $p(\theta, z \mid y)$.



6.4 No-U-Turn Sampler (NUTS)

An important hyper parameter in HMC that we need to specify is the number of leapfrog steps L : if L is too small, the chain does not move far and it takes us a long time to explore the state space. If L is large, do we always explore more?

No! We may loop around and come back close to the starting state, i.e., the trajectory can loop back and “undo” progress (see the figure below). The *No-U-Turn Sampler (NUTS)* adapts the trajectory length L automatically.

Main idea. NUTS extend the trajectory forward and backward in time until a *U-turn* is detected—the momentum begins to point back toward the starting position. Then it samples a state from the built trajectory in a way that preserves detailed balance. NUTS chooses η and Σ adaptively during the burn-in period, and chooses L adaptively at each iteration.

NUTS is the standard MCMC algorithm used in Stan, described in the next section.

7 A Brief Introduction to Stan

Stan (<https://mc-stan.org>) is a probabilistic programming language for *full Bayesian inference* in models with continuous parameters. Under the hood, Stan uses adaptive *No-U-Turn Sampler (NUTS)*, an extension of HMC, together with automatic differentiation to compute gradients of the log posterior. Compared with earlier tools such as WinBUGS/JAGS, Stan often mixes faster in moderate-to-high dimensions and provides richer diagnostics.

Interfaces and installation. Stan can be used (i) as a stand-alone command-line tool, or (ii) via language interfaces such as RStan (R), CmdStanR (R), and PyStan (Python). In this course we will use RStan. Installation instructions:

<https://mc-stan.org/users/interfaces/>

(See also the RStan getting-started guide linked from that page: <https://github.com/stan-dev/rstan/wiki/RStan-Getting-Started#prerequisites>.) Python (PyStan) and MATLAB (MatlabStan) interfaces are available as well. So, you are definitely not restricted to use R.

To use Stan we need to

1. write a code (stan program) to specify a Bayesian statistical model (stan model),
2. then call stan with appropriate data to fit the model.

How Stan programs are structured. A Stan model is a short program in blocks. The most common blocks are:

- **data block:** declare observed inputs (sizes/types only).
- **parameters block:** declare unknowns to infer (with constraints).
- **model block:** define the prior and likelihood (contributing to the target density).

Another very useful optional block is:

- **generated quantities:** posterior predictions, log-likelihoods, and any derived quantities computed *after* sampling (no effect on sampling).

7.1 A first example: the Normal model

Data in R. First, we generate and prepare the data in R.

```
# generate data
set.seed(123)
N <- 100
Y <- rnorm(N, mean = 2, sd = 0.3)
```

Stan program (model.stan). Then we start a .stan file in R and write the Stan program. Below is a minimal model with weakly informative priors. Note that `sigma` has a positivity constraint; `cauchy(0,1)` on a `<lower=0>` parameter induces a *half-Cauchy* prior.

```
data {
  int<lower=1> N;
  vector[N] Y;
}

parameters {
  real mu;
  real<lower=0> sigma;
}

model {
  // priors
  mu ~ normal(1.5, 0.5);
  sigma ~ cauchy(0, 1);      // half-Cauchy on (0, infinity)
                             (Stan normalizes automatically)
  // sampling model (vectorized)
  Y ~ normal(mu, sigma);
}
```

Note that if you are using Python or other languages, the above code remains unchanged.

Compiling and sampling in R with RStan. Next we use the package `rstan` to compile the model. We need to first install `rstan`.

```
library(rstan)
options(mc.cores = parallel::detectCores()) # run chains in parallel

mod <- stan_model('model.stan')           # compile
fit <- sampling(mod,
  data = list(N = N, Y = Y),
  iter = 2000, warmup = 1000, chains = 4, seed = 2024,
  control = list(adapt_delta = 0.9, max_treedepth = 12))
```

Notes.

- **Warmup (a.k.a. burn-in)** is used for adapting step size and mass matrix.
- **Control parameters:** `adapt_delta` (target acceptance, closer to 1 reduces divergences but may slow sampling) and `max_treedepth` (limits trajectory length).

Remark. You can alternatively use `stan(file = "...", ...)`, which not only samples, but also compiles

Inspecting results and diagnostics. The object `fit` contains

- Diagnostics
- Posterior samples

```
print(fit, digits_summary = 2)      # posterior summaries, R-hat, ESS

# Trace and ACF plots
stan_trace(fit, pars = c("mu","sigma"))
stan_ac(fit,    pars = c("mu","sigma"))

# Posterior densities / histograms
stan_dens(fit,  pars = c("mu","sigma"))
stan_hist(fit,  pars = c("mu","sigma"))
```

- The `print` output reports the information regarding the process such as number of chains and the burn-in period, as well as summary statistics for the parameters such as posterior mean/SD, Monte Carlo error, effective sample size (bulk/tail), and $\hat{R}$ (should be very close to 1). Watch for HMC-specific diagnostics in the sampler messages.
- You can perform convergence diagnostics for instance using `stan_trace(fit)` for trace plot for the chains and `stan_ac(fit)` for auto-correlation function plot. Also, you can look at the posterior densities and histograms using `stan_dens(fit)` and `stan_hist(fit)`, respectively.

Extracting draws and post-processing. We can *extract* the parameters as follows:

```
post <- rstan::extract(fit) # list of posterior draws by parameter
str(post$mu)               # numeric vector of draws for mu
str(post$sigma)
```

Alternatively, `as.data.frame(fit)` or `as.array(fit)` provide tidy formats for use with `bayesplot/posterior` packages.

Adding posterior predictions (generated quantities) It is often useful to simulate replicated data and compute derived quantities without affecting sampling:

```
generated quantities {
  vector[N] y_rep;
  for (n in 1:N) y_rep[n] = normal_rng(mu, sigma);
}
```

After sampling, extract `y_rep` to perform posterior predictive checks.

Good practices and common gotchas

- **Vectorize and use built-in lpdf/rng functions.** They are faster and more stable than explicit loops.
- **Constrain parameters** (e.g., `<lower=0>`) to reflect support; transform constrained models to $\mathbb{R}^d$ automatically.
- **Check diagnostics first:**
Divergences $\Rightarrow$ reduce `stepsize` / increase `adapt_delta`;
Treedepth hits $\Rightarrow$ increase `max_treedepth`.
- **Reproducibility.** Set an RNG `seed`; record `iter`, `warmup`, `chains`, and `control`.

Remark. An amazing tool for posterior analysis and diagnostics is the package `shinystan`:

```
library(shinystan)
launch_shinystan(fit)
```

Specifically, look at the diagnose tab for trace plot, auto-correlation function plots, histogram of samples, acceptance rates, comparison of the chains, etc.

Further resources. One of the nicest features of `stan` is its wonderful documentations. Take a look at *Stan User's Guide* in the following link. It provides several examples, discussions for several important classes of models, modeling tips, and diagnostics.

<https://mc-stan.org/docs/stan-users-guide/index.html>

7.2 Linear Regression with Stan

We illustrate Bayesian simple linear regression in `Stan` using the classic `iris` dataset. Iris dataset gives the measurements (in centimeter) of the sepal length and width and petal length and width for 50 flowers from each of 3 species of iris. The species are *setosa*, *versicolor*, and *virginica*.

We are interested in identifying a linear relation between the sepal length and the petal length of the versicolor irises. Hence, we will regress *petal length* (response) on *sepal length* (predictor) for the versicolor subset.

Prepare the data in R. First, we prepare the data. Remember that Stan receives data as a list.

```
data(iris)
head(iris)

iris_v <- subset(iris, Species == "versicolor")
x <- iris_v$Sepal.Length           # predictor
y <- iris_v$Petal.Length           # response
N <- length(y)
data_list <- list(N = N, x = as.vector(x), y = as.vector(y))
```

Tip. In practice, centering/scaling predictors (e.g., $x_c = x - \bar{x}$) improves geometry (reduces posterior correlation between slope and intercept) and can speed HMC. We keep the raw scale here for interpretability.

Stan model (linreg.stan). We place weakly informative priors on the intercept α , slope β , and noise scale $\sigma > 0$. Because of the `<lower=0>` constraint, `sigma ~ normal(0,1)` implies a *half-normal* prior.

```
data {
  int<lower=1> N;           // number of observations
  vector[N] x;             // predictor (sepal length)
  vector[N] y;             // response (petal length)
}

parameters {
  real alpha;              // intercept
  real beta;               // slope
  real<lower=0> sigma;      // residual SD
}

model {
  // weakly informative priors
  alpha ~ normal(0, 10);
  beta ~ normal(0, 10);
  sigma ~ normal(0, 1);    // half-N(0,1) due to lower=0

  // sampling model (vectorized)
  y ~ normal(alpha + beta * x, sigma);
}

generated quantities {
  // posterior predictive for each observed x (useful for checks/plots)
  vector[N] y_rep;
```

```

  for (n in 1:N) y_rep[n] = normal_rng(alpha + beta * x[n], sigma);
}

```

Note that we are not using semiconjugate prior nor g -prior.

Fit the model from R with RStan.

```

library(rstan)
options(mc.cores = parallel::detectCores())

fit <- stan(file = "linreg.stan",
            data = data_list,
            chains = 4, iter = 4000, warmup = 2000, seed = 2025,
            control = list(adapt_delta = 0.9, max_treedepth = 12))

print(fit, pars = c("alpha","beta","sigma"),
      probs = c(0.025, 0.5, 0.975))

```

The printed summary reports posterior means/quantiles and key diagnostics: $\hat{R}$ (target ≈ 1), effective sample sizes, and sampler messages (e.g., divergent transitions). Increase `adapt_delta` (e.g., to 0.95–0.99) if you see divergences.

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
alpha	0.19	0.01	0.52	-0.83	-0.16	0.18	0.54	1.20	2269	1
beta	0.69	0.00	0.09	0.52	0.63	0.69	0.75	0.86	2260	1
sigma	0.32	0.00	0.03	0.26	0.30	0.32	0.34	0.39	2567	1

Diagnostics and basic plots.

```

stan_trace(fit, pars = c("alpha","beta","sigma")) # trace plots
stan_ac(fit,   pars = c("alpha","beta","sigma")) # autocorrelation
stan_dens(fit, pars = c("alpha","beta","sigma")) # marginal posteriors

```

Extract draws for custom summaries/plots:

```

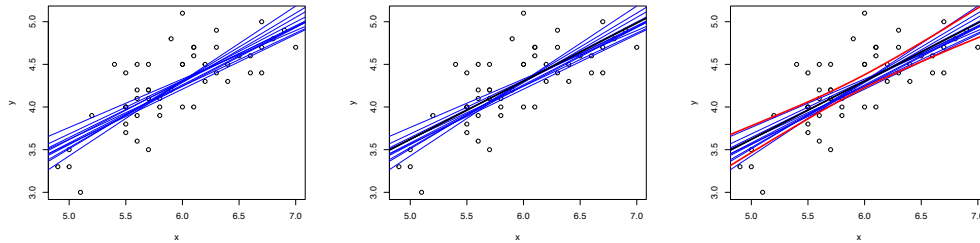
post <- rstan::extract(fit)
alpha_draws <- post$alpha
beta_draws <- post$beta
sigma_draws <- post$sigma
yrep_draws <- post$y_rep # N x (iterations) in array/list form

```

Interpreting the fitted line and uncertainty bands. Let $\hat{\alpha} = \mathbb{E}[\alpha \mid y]$ and $\hat{\beta} = \mathbb{E}[\beta \mid y]$. The line $\hat{y}(x) = \hat{\alpha} + \hat{\beta}x$ summarizes the *posterior mean* regression. Two useful envelopes are:

- a 95% credible band for the mean function $m(x) = \alpha + \beta x$, obtained by computing $\alpha^{(s)} + \beta^{(s)}x$ across draws; and
- a 95% posterior predictive band for new y at x , which adds residual uncertainty via $y_{\text{new}}^{(s)} \sim \mathcal{N}(\alpha^{(s)} + \beta^{(s)}x, \sigma^2(s))$.

The figures below show (i) sample regression lines from posterior draws, (ii) the fitted mean line based on $(\hat{\alpha}, \hat{\beta})$, i.e., the fitted line based on the posterior mean of the intercept and the coefficient, and (iii) 95% quantile-based credible band constructed from the draws.



Remark (Centering for better geometry). Centering x as $x_c = x - \bar{x}$ (and using x_c in the model) often increases effective sample size by reducing posterior correlation between α and β . When reporting results, transform back to the original scale: $\alpha_{\text{orig}} = \alpha - \beta \bar{x}$, $\beta_{\text{orig}} = \beta$.

What we *did not* use here. For pedagogy we chose simple weakly informative priors. We did not use semi-conjugate or g -priors; both can be encoded in **Stan**, but are unnecessary for efficient sampling in this simple model.

Remark: rstanarm for quick modeling. **rstanarm** is an R package that emulates other R model-fitting functions but uses Stan (via the **rstan** package) for the back-end estimation. The primary target audience is people who would be open to Bayesian inference if using Bayesian software were easier but would use frequentist software otherwise.

For linear models:

- `stan_lm` mirrors `lm` function in syntax, but rather than choosing the parameters to minimize the sum of squared residuals, samples from the posterior distribution are drawn using MCMC (if algorithm is “sampling”).
- `stan_glm(family = "gaussian")` also estimates a linear model with normally-distributed errors and allows for various other priors on the coefficients.

These interfaces are convenient when you want Bayesian inference with familiar R syntax and sensible default priors, without writing a Stan program.